

**Reference
Notes
[Digital Copy]**



MongoDB Database

Join us for Better Career and Job Oriented Training

[Live Online & Offline Classes Available]

Vision

To be an institute that provides a transformative learning to produce highly skilled & competent professionals and to create leaders and innovators for society and industry.

Mission

- To provide quality education and learning solutions to students and working professionals in order to enhance their practical knowledge, skills, and working experience.
- To provide an environment for high-quality education and outcome based training to students and working professionals, equipping them with the practical knowledge, skills, and attitude required to succeed in their personal and professional lives.
- To deliver practical, hands-on training services that prepares our students according to the real-world challenges and opportunities, while fostering a culture of learning, execution and continuous improvement.

Director(s)



Mr. Mohit Singh

M.Tech, B.Tech (C.S.E)

Mr. Mohit Singh is a professional full-stack trainer, project consultant and startup mentor. He is holding expertise in Java, Application Design, MERN Stack, Data Science & ML, DevOps, Design Thinking, User Experience Design, etc.

He has trained thousands of students & hundreds of employed professionals. He completed his trainings in Google, Gurugram and short term projects in IIT Delhi, IIT BHU & IIT Jodhpur.

He is also recognized as Mentor with startup India, Punjab Startup, startup Uttarakhand, Mumbai State Innovation Society, Atal Innovation Mission, etc. in the area of Education & Utility services.



<https://www.linkedin.com/in/mohit9pages/>



Dr. Shubhendra Gupta

Phd, B.Ed, M.Sc (Physics)

Dr. Shubhendra Gupta is an experienced digital marketer, business consultant and startup mentor with a demonstrated history of working in the education and services industry.

He use to train students & working professionals for getting better job opportunities and train business owners in generating profits or leads. His areas of interest are Digital Marketing, Business Development, Data Analysis, Strategic Planning, Market Research & Reality, User Testing, Website design, etc.

He is also recognized as Mentor with Startup Hubs & Innovation Labs in the area of education, brand building, business development & consultation.



<https://www.linkedin.com/in/dmshubhendra/>

Disclaimer

At **Learn2Earn Labs**, we believe that learning should be a journey of growth, confidence, and career excellence. The notes we provide are carefully designed to guide students on **what to learn** and **how to learn** so they can build strong expertise, improve practical skills, and unlock better career opportunities.

We put our heart and effort into creating these resources with the sole objective of helping learners reach their full potential. While every topic is prepared with great care, we acknowledge that—being human—there may occasionally be minor errors, incomplete details, or areas that need refinement.

If you ever come across such points, we warmly encourage you to write to us at **learn2earnlabs@gmail.com** or **team@learntoearnlabs.com**. Your feedback not only helps us correct and improve, but also strengthens our mission of providing **the best quality education and guidance** to every learner.

These notes are meant to serve as a **direction and roadmap**, not the final destination. We encourage you to use them as a stepping stone—explore more, practice consistently, and challenge yourself to go beyond. With dedication, the right path, and our continuous support, you can transform yourself into a true professional and achieve the success you deserve.

MongoDB

MongoDB is a powerful and flexible database system that offers scalability, performance, and flexibility, making it well-suited for a wide range of use cases, from web and mobile applications to real-time analytics and content management systems.

Here's a brief introduction to MongoDB:

- **NoSQL Database:** MongoDB falls under the category of NoSQL databases, which means it doesn't rely on the traditional tabular relations used in relational databases like SQL. Instead, MongoDB uses a flexible, document-oriented data model that can store data in JSON-like documents with dynamic schemas.
- **Document-Oriented:** In MongoDB, data is stored in documents, which are essentially JSON-like data structures composed of field-value pairs. These documents are organized into collections, which are similar to tables in relational databases but are schema-less and can store documents with varying structures.
- **Scalability:** MongoDB is designed to scale horizontally across multiple servers, making it suitable for handling large volumes of data and high-traffic applications. It supports sharding, which involves distributing data across multiple machines, and replica sets, which provide redundancy and high availability.
- **High Performance:** MongoDB is optimized for high performance, with features like indexing, query optimization, and support for in-memory computing. It also offers features like aggregation pipelines for complex data processing tasks.
- **Query Language:** MongoDB provides a rich query language that supports a wide range of operations for querying and manipulating data. It supports CRUD operations (Create, Read, Update, Delete), as well as advanced operations like aggregation, text search, geospatial queries, and more.
- **Flexible Schema:** Unlike relational databases with fixed schemas, MongoDB offers a flexible schema that allows documents in the same collection to have different structures. This flexibility makes it easier to iterate on data models and accommodate changes in application requirements over time.
- **Community and Ecosystem:** MongoDB has a large and active community of developers and contributors, as well as a rich ecosystem of tools, libraries, and services built around it. This ecosystem includes tools for data visualization, monitoring, management, and integration with other technologies.

MongoDB was created by a company '10Gen', which is now known as MongoDB Inc.

NoSQL VS SQL

let's discuss the differences between SQL (relational databases) and NoSQL databases in detail, along with suitable examples:

1. Data Model

- **SQL (Relational Databases)**

- Relational databases follow a structured data model based on tables with rows and columns.
- Data is organized into tables, each defining a specific entity, with relationships established through foreign keys.
- **Examples:** MySQL, PostgreSQL, SQLite.

- **NoSQL Databases**

- NoSQL databases offer various data models, including document-based, key-value pairs, column-oriented, and graph databases.
- Each NoSQL database type provides a different way of structuring and organizing data, offering flexibility in data modeling.
- **Examples:** MongoDB (document-based), Redis (key-value pairs), Cassandra (column-oriented), Neo4j (graph database).

2. Schema

- **SQL (Relational Databases)**

- Relational databases enforce a fixed schema, where the structure of data (tables, columns, data types) must be predefined before inserting data.
- Changes to the schema typically require altering the database schema, which can be complex and time-consuming.

- **NoSQL Databases**

- NoSQL databases are schema-less or have a flexible schema, allowing for dynamic changes to the data structure without requiring a predefined schema.
- Each document or record in a NoSQL database can have its own structure, enabling easier adaptation to evolving application requirements.

3. Query Language

- **SQL (Relational Databases)**
 - SQL (Structured Query Language) is the standard language for querying and manipulating data in relational databases.
 - SQL provides a declarative syntax for performing CRUD operations (Create, Read, Update, Delete), as well as complex queries and joins.
- **NoSQL Databases**
 - NoSQL databases typically have their own query languages optimized for specific data models.
 - Query languages may vary between different NoSQL databases but generally offer capabilities for basic CRUD operations and specific querying features tailored to the database type.

4. Scalability

- **SQL (Relational Databases)**
 - Relational databases are traditionally scaled vertically (scale-up), by adding more resources (CPU, memory) to a single server.
 - Horizontal scaling (scale-out) can be challenging and may involve complex partitioning strategies.
- **NoSQL Databases**
 - NoSQL databases are designed for horizontal scalability, allowing data to be distributed across multiple servers or nodes.
 - NoSQL databases can handle large volumes of data and high throughput by adding more nodes to the cluster, without requiring significant changes to the application or database architecture.

5. Examples

- **SQL (Relational Databases)**
 - **Example:** Consider an e-commerce app storing data about customers, orders, & products in relational tables (e.g., Customers, Orders, Products).
 - Tables would have predefined schemas with fixed columns, and relationships between tables would be established through foreign keys.

- **NoSQL Databases**

- **Example:** In the same e-commerce application, using MongoDB as a NoSQL database, each order could be represented as a JSON document within a collection.
- The structure of the order document could vary based on the items ordered, without requiring a fixed schema for all orders.
- This flexibility allows for easier handling of nested data, such as customer details and order items, within a single document.

Hence, SQL databases (relational databases) and NoSQL databases differ in their data models, schema flexibility, query languages, scalability approaches, and usage scenarios. The choice between SQL and NoSQL databases depends on factors such as data structure, scalability requirements, and application complexity.

Use-Cases : NoSQL

NoSQL databases are often the best option to use in the following scenarios:

- **Handling Large Volumes of Data:** NoSQL databases are designed to scale horizontally, making them well-suited for handling large volumes of data distributed across multiple servers or nodes. They can efficiently manage big data workloads without sacrificing performance.
- **Flexible Schema Requirements:** NoSQL databases offer flexible schema options, allowing developers to store data with varying structures within the same collection or table. This flexibility is beneficial for applications with evolving or unpredictable data schemas.
- **High Availability and Fault Tolerance:** NoSQL databases, particularly those with features like automatic failover and replication (e.g., MongoDB replica sets), provide high availability and fault tolerance. They can automatically recover from hardware failures and ensure continuous operation.
- **Scalability Requirements:** NoSQL databases excel at horizontal scalability, allowing for seamless scaling out by adding more nodes to the database cluster. This makes them suitable for applications with growing data and user loads, without the need for complex partitioning strategies.
- **Real-time Analytics and IoT Applications:** NoSQL databases are commonly used in real-time analytics and IoT (Internet of Things) applications, where data is

generated and processed rapidly. Their ability to handle high-throughput workloads and semi-structured data makes them ideal for capturing and analyzing real-time data streams.

- **Document-centric Applications:** NoSQL document databases like MongoDB are particularly well-suited for document-centric applications, such as content management systems, e-commerce platforms, and social media applications. They allow developers to store and query complex, nested data structures (e.g., JSON documents) efficiently.
- **Agile Development and Rapid Prototyping:** NoSQL databases offer schema flexibility and agile development capabilities, allowing developers to iterate quickly on data models and application features. They are conducive to agile development practices, enabling faster time-to-market for new applications and features.
- **Geospatial and Graph Data:** Some NoSQL databases provide specialized features for handling geospatial data (e.g., MongoDB's geospatial indexes) or graph data (e.g., Neo4j). These features are beneficial for applications requiring spatial queries, location-based services, or graph-based data modeling.

Hence, NoSQL databases are a preferred choice for applications requiring scalability, flexibility, high availability, and real-time data processing capabilities. They are well-suited for modern, data-intensive applications across various industries, including e-commerce, gaming, social networking, IoT, and analytics.

Aspects of MongoDB

MongoDB is a feature-rich and versatile database system that offers scalability, flexibility, performance, and robustness, making it well-suited for modern applications across various industries and use cases.

- **Data Model:** MongoDB stores data in flexible, JSON-like documents called BSON (Binary JSON), which allows for a more natural representation of hierarchical relationships and complex data structures. Each document can have a different structure, with varying fields and data types, within the same collection.
- **Collections and Documents:** MongoDB organizes data into collections, which are analogous to tables in relational databases. Each collection contains multiple documents, and documents within a collection can have different structures. Collections and documents are schema-less, meaning there is no requirement for predefined schemas or data types.
- **Indexes:** MongoDB supports various types of indexes to improve query performance. Indexes can be created on single fields, compound fields, arrays, and even text fields for full-text search. Indexes are essential for efficiently querying large datasets and ensuring fast data retrieval.
- **Replication:** MongoDB provides replication functionality through replica sets, which are self-healing clusters of MongoDB servers. Replica sets maintain multiple copies of data across different servers to ensure data redundancy, fault tolerance, and high availability. If one server fails, another server can automatically take over to ensure continuous operation.
- **Sharding:** Sharding is MongoDB's approach to horizontal scaling, allowing the distribution of data across multiple servers or shards. Sharding improves scalability by distributing data and query load evenly across multiple machines. MongoDB's sharding architecture is based on a sharded cluster, which consists of multiple shards, config servers, and query routers (mongos).
- **Query Language and Aggregation:** MongoDB's query language is rich and expressive, supporting a wide range of operations for querying and manipulating data. It includes CRUD operations (Create, Read, Update, Delete), as well as advanced query features such as aggregation pipelines, which allow for complex data transformations, grouping, filtering, and analysis directly within the database.
- **Transactions:** MongoDB introduced multi-document transactions in version 4.0, allowing developers to perform atomic operations across multiple documents within a single transaction. This feature ensures data integrity and consistency,

especially in scenarios where operations across multiple documents need to be coordinated.

- **Security:** MongoDB provides various security features to protect data, including authentication, authorization, encryption, and auditing. It supports role-based access control (RBAC), allowing administrators to define granular access controls and permissions for users and applications accessing the database.
- **Scalability and Performance:** MongoDB is designed for high performance and scalability, with features such as in-memory computing, automatic load balancing, and horizontal scaling through sharding. It is optimized for both read and write-intensive workloads, making it suitable for a wide range of applications with diverse performance requirements.
- **Community and Support:** MongoDB has a vibrant and active community of developers, users, and contributors. It offers comprehensive documentation, online forums, user groups, and official support options to help users get started, troubleshoot issues, and optimize their MongoDB deployments.

Best Fit Consideration: MongoDB

- MongoDB is a versatile database system that can be used in a wide range of projects and software applications, including content management systems, e-commerce platforms, real-time analytics, mobile app backends, IoT, gaming, and location-based services.
- Its flexibility, scalability, performance, and ease of development make it a preferred choice for many developers and organizations across various industries.
- MongoDB is well-suited for a wide range of projects and software applications, particularly those that benefit from its flexible data model, scalability, performance, and ease of development.

Here are some types of projects and applications for which MongoDB is often considered the best fit:

- **Content Management Systems (CMS):** MongoDB's document-oriented data model is well-suited for managing content in CMS applications. It allows for flexible schema design, making it easy to store and retrieve structured and unstructured content, such as articles, blog posts, images, and videos.

- **E-commerce Platforms:** MongoDB is suitable for e-commerce applications, where product catalogs, customer data, and order information need to be stored and managed efficiently. Its scalability and performance make it ideal for handling large volumes of product listings, customer transactions, and inventory data.
- **Real-time Analytics and Big Data:** MongoDB is commonly used in real-time analytics and big data applications, where data is ingested, processed, and analyzed in real-time. Its ability to handle high-speed data ingestion and complex queries makes it well-suited for applications that require real-time insights and data-driven decision-making.
- **Social Media Platforms:** MongoDB is a popular choice for building social media platforms and networking applications. Its flexible data model allows developers to store user profiles, social interactions, messages, and multimedia content in a scalable and efficient manner.
- **Mobile App Backends:** MongoDB serves as an excellent backend database for mobile applications, providing offline synchronization, geospatial queries, and real-time data updates. Its support for JSON-like documents and flexible schema design simplifies data synchronization between mobile devices and backend servers.
- **Internet of Things (IoT):** MongoDB is well-suited for IoT applications, where sensor data, telemetry, and device information need to be stored and analyzed in real-time. Its ability to handle time-series data, geospatial queries, and complex event processing makes it an ideal choice for IoT data management.
- **Gaming Platforms:** MongoDB is commonly used in gaming platforms and online gaming applications. Its scalability, performance, and support for real-time data updates make it suitable for managing player profiles, game states, leaderboards, and in-game transactions.
- **Location-based Services:** MongoDB's support for geospatial indexes and queries makes it suitable for location-based services and geospatial applications. It can efficiently store and query geospatial data, such as maps, points of interest, and location-based user interactions.
- **Prototyping and Rapid Development:** MongoDB's schema flexibility, agile development capabilities, and ease of use make it an ideal choice for prototyping and rapid development projects. It allows developers to iterate quickly on data models and application features, accelerating the development process.

Join us for Industry Based & Job Oriented Training Programs

Top Edge Training Programs



**MERN / MEAN Stack
Development**



**Java Full Stack
Development**



**Data Science
& Machine Learning**



**Data Analytics &
Business Intelligence**



**Linux, Cloud
& DevOps**



**Digital Marketing &
Business Development**

Amenities

- Digital Notes
- Learning References
- Live Training Sessions
- Live Project Training
- Interview Preparation
- Live Session Recordings for Revision
- Job Referrals & Recommendations
- Project Work Experience
- Professional Development
- Digital Resume & Portfolio

LEARN-2-EARN LABS TRAINING INSTITUTE

Head Office : F-4, First Floor, Anna Icon Complex, near Kargil Petrol Pump,
Sikandra-Bodla Road, Sikandra, Agra, Uttar Pradesh - 282007

Website : www.LearntoearnLabs.com

Call : 91-9548868337 / +91-9837705705

JSON Vs BSON

- JSON is a human-readable, text-based data interchange format widely used for transmitting and storing structured data, while BSON is a binary serialization format optimized for storage and transmission efficiency in MongoDB.
- JSON is commonly used in web development and APIs, while BSON is specific to MongoDB and is used internally for storing documents in the database.
- Both JSON (JavaScript Object Notation) and BSON (Binary JSON) are data interchange formats used to represent structured data.

JSON (JavaScript Object Notation)

- JSON is a lightweight, text-based data interchange format that is easy for humans to read and write, and easy for machines to parse and generate.
- It is commonly used for transmitting data between a web server and a web client, as well as for storing configuration settings and exchanging data between systems.
- JSON data is represented as key-value pairs, where keys are strings and values can be strings, numbers, booleans, arrays, or nested objects.
- **Format:** JSON is a lightweight, text-based data interchange format inspired by JavaScript object syntax.
- **Human-readable:** JSON is easy for humans to read and write, and it's also easy for machines to parse and generate.

Example of JSON data:

```
{  
  "name": "Nikhil garg",  
  "age": 14,  
  "languages": ["JavaScript", "React", "Node"],  
  "address": {  
    "city": "Agra",  
    "zipcode": "10001"  }  
}
```

- **Data Types:** JSON supports several data types, including:
 - **Objects:** Key-value pairs enclosed in curly braces {}.
 - **Arrays:** Ordered lists of values enclosed in square brackets [].
 - **Strings:** Sequences of characters enclosed in double quotes " " or single quotes ' '.
 - **Numbers:** Integer or floating-point numbers.
 - **Booleans:** true or false.
 - **Null:** null.

BSON (Binary JSON)

- BSON is a binary serialization format used to store and transfer data in MongoDB, a NoSQL database system.
- Unlike JSON, which is text-based, BSON is a binary format, which makes it more compact and efficient for storage and transmission.
- BSON extends JSON by adding additional data types such as Date, Binary Data, and ObjectId, which are commonly used in MongoDB documents.
- BSON documents have a similar structure to JSON documents, with key-value pairs representing the data fields, but they are encoded in binary format for storage and transmission efficiency.
- **Format:** BSON is a binary serialization format used primarily in MongoDB for storing and transferring data.
- **Efficiency:** BSON is designed to be more compact and efficient than JSON, making it suitable for storage and transmission over networks.

Example of BSON data (same as JSON but represented in BSON format):

```
{
  "name": "Nikhil garg",
  "age": 14,
  "languages": ["JavaScript", "React", "Node"],
  "address": {
    "city": "Agra",
    "zipcode": "10001"
  }
}
```

- **Data Types:** BSON extends JSON by adding additional data types, including:
 - **Date:** Represents date and time information.
 - **Binary Data:** Represents binary data in various formats.
 - **ObjectId:** Represents a unique identifier used in MongoDB.
 - **Regular Expression:** Represents regular expression patterns.
 - **Timestamp:** Represents a time stamp.
 - **Decimal128:** Represents high-precision floating-point numbers.

BSON documents consist of a series of fields and values encoded in a binary format.

For example:

```
\x31\x00\x00\x00\x02hello\x00\x06\x00\x00\x00world\x00\x00
```

BSON is specific to MongoDB and is used internally for storing documents in collections. It allows MongoDB to efficiently store and retrieve data, perform queries, and manage indexes.

\x31\x00\x00\x00\x02hello\x00\x06\x00\x00\x00world\x00\x00

The string `\x31\x00\x00\x00\x02hello\x00\x06\x00\x00\x00world\x00\x00` is a representation of a BSON document encoded in hexadecimal. Each pair of hexadecimal digits represents a byte in the BSON data.

Let's break down the structure of this BSON document:

- **\x31\x00\x00\x00**: This represents the total size of the BSON document, including itself. The total size is 49 bytes (\x31 in hexadecimal is 49 in decimal).
- **\x02**: This byte represents the data type of the field "hello". In BSON, 0x02 represents a string type.
- **hello\x00**: This is the field name. The string "hello" followed by a null terminator (\x00).
- **\x06\x00\x00\x00**: This represents the length of the string value "world". The length is 6 bytes (\x06\x00\x00\x00 in little-endian byte order).
- **world\x00**: This is the string value "world" followed by a null terminator (\x00).
- **\x00**: This marks the end of the BSON document.

Hence, the string `\x31\x00\x00\x00\x02hello\x00\x06\x00\x00\x00world\x00\x00` represents a BSON document with a single field named "hello" with the value "world". The total size of the BSON document is 49 bytes.

LABS

Don't Forget

- MongoDB is designed for flexibility, scalability, high performance and handling unstructured & semi-structured data.
- Unstructured and semi-structured data are two types of data formats that do not fit neatly into traditional structured formats, such as rows and columns in a relational database.
- Unstructured data lacks a predefined structure and is often challenging to process, while semi-structured data retains some level of organization or metadata, making it more accessible but still requiring preprocessing for analysis.
- Both types of data are common in modern applications and require specialized tools and techniques for effective management and analysis.

Unstructured Data

- **Definition:** Unstructured data refers to data that does not have a predefined data model or organization. It lacks a formal structure, making it challenging to process and analyze using traditional database management systems.
- **Characteristics**
 - Unstructured data can include text documents, images, audio files, video files, social media posts, emails, sensor data, and more.
 - It often contains natural language text, multimedia content, or raw data without a specific schema or format.
 - Unstructured data is typically stored in file systems, content management systems, NoSQL databases, or data lakes.
- **Examples**
 - Text documents (e.g., Word documents, PDFs)
 - Images (e.g., photographs, scanned documents)
 - Audio files (e.g., voice recordings, podcasts)
 - Video files (e.g., movies, surveillance footage)
- **Challenges**
 - Extracting meaning and insights from unstructured data requires advanced techniques such as natural language processing (NLP), image processing, audio analysis, and machine learning.
 - Unstructured data can be voluminous and difficult to manage, requiring specialized storage and processing solutions.

Semi-Structured Data

- **Definition:** Semi-structured data refers to data that does not conform to a rigid structure like relational databases but contains some level of organization or metadata that enables partial understanding.
- **Characteristics**
 - Semi-structured data retains some form of organizational structure or metadata, making it more accessible than unstructured data.
 - It may have a flexible schema or a loosely defined schema, allowing for variations in data representation within the same dataset.
 - Semi-structured data often includes formats such as JSON (JavaScript Object Notation), XML (eXtensible Markup Language), YAML (YAML Ain't Markup Language), CSV (Comma-Separated Values), and log files.
- **Examples**
 - JSON documents (e.g., API responses, NoSQL databases)
 - XML files (e.g., RSS feeds, configuration files)
 - CSV files (e.g., spreadsheets, tabular data)
 - Log files (e.g., server logs, application logs)
- **Advantages**
 - Semi-structured data offers a balance between flexibility and organization, making it suitable for scenarios where data schemas may evolve over time.
 - It can be processed using a combination of traditional database techniques and modern data processing tools like Apache Spark, Hadoop, and Elasticsearch.
- **Challenges**
 - Semi-structured data still requires preprocessing and transformation to extract meaningful insights, particularly when dealing with nested or complex structures.
 - Managing schema evolution and data quality can be challenging, especially in large and heterogeneous datasets.

Inside MongoDB database, we have collections and inside every collections, we have documents which is in json/bson format.

MongoDB Architecture

In MongoDB, the server, storage engine, and database work together to store, manage, and access data efficiently.

How they work

- **MongoDB Server**

- The MongoDB server (mongod) is responsible for handling client connections, executing database operations, and coordinating data storage and retrieval.
- The server process listens for incoming client requests over the network and communicates with client applications using the MongoDB wire protocol.
- The server manages multiple databases and collections, each containing documents organized in BSON format.
- The server supports various administrative operations, including creating databases, collections, and indexes, as well as managing user authentication and authorization.

- **Storage Engine**

- The storage engine is responsible for managing data storage, indexing, and retrieval on disk. MongoDB supports multiple storage engines, including WiredTiger (default) and MMAPv1 (deprecated).
- The storage engine handles data persistence, ensuring that data modifications are durably stored on disk and can be recovered in case of server failures or crashes.
- It manages storage structures such as data files, journal files, indexes, and in-memory caches to optimize data access and performance.
- The storage engine implements data compression, concurrency control, and transaction management to ensure data consistency and integrity.

- **Database**

- A database in MongoDB is a logical container for storing collections of documents. Each database is identified by a unique name and can contain multiple collections.
- Databases are created dynamically as needed and do not require predefined schemas or data types. MongoDB supports flexible schema design, allowing documents within the same collection to have different structures.
- Each database has its own set of users and privileges, allowing for fine-grained access control and security policies.

Hence, you can understand that

- The MongoDB server acts as the core component responsible for handling client connections, executing database operations, and managing data storage and retrieval.
- The storage engine is responsible for managing data storage on disk and optimizing data access and performance.
- Databases provide logical containers for organizing collections of documents and support flexible schema design and fine-grained access control. MongoDB databases are automatically sharded and distributed across multiple nodes in sharded clusters for horizontal scalability and fault tolerance.

Together, these components work together to provide a flexible, scalable, and high-performance database system for modern applications.

MongoDB Installation & Setup

- MongoDB Community Server

<https://www.mongodb.com/try/download/community>

through the setup installation, don't forget to check "Install MongoDB Compass"

- MongoDB Shell

<https://www.mongodb.com/try/download/shell>

- MongoDB Command Line Database Tools

<https://www.mongodb.com/try/download/database-tools>

- Now add the following path to System Environment variables

<C:\Program Files\MongoDB\Server\7.0\bin>

- To check the version of mongodb server, use following command in command prompt

`Mongod -version`

Now we don't need to start/run the mongodb server manually because its automatically gets started. You can check it by your system's task manager processes log.

Using shell for execution

- Use "**mongosh**" command in command prompt to enter inside mongodb shell.
- To check the available database, use command
show dbs
or show databases
- To create a database in mongodb, or to switch between databases, use command
use 'database_name'
for ex: use emp_records

now If you want to see the available databases, using "showdbs" command then you will not get the name of newly created database, because no any collection or document is available inside the database.

So, we need to create a collection or document inside the newly created database.

- To create collection inside the database, use command:
use database_name
db.createCollection('collection_name') //to create a single collection
for ex: db.createCollection('emp_data')
- To check the available connection inside the database, use command
show collections
- To delete the available collection inside the database, use command
db.emp_data.drop()
- To delete the database, use command
db.dropDatabase()

Join us for Industry Based & Job Oriented Training Programs

Top Edge Training Programs



**MERN / MEAN Stack
Development**



**Java Full Stack
Development**



**Data Science
& Machine Learning**



**Data Analytics &
Business Intelligence**



**Linux, Cloud
& DevOps**



**Digital Marketing &
Business Development**

Amenities

- Digital Notes
- Learning References
- Live Training Sessions
- Live Project Training
- Interview Preparation
- Live Session Recordings for Revision
- Job Referrals & Recommendations
- Project Work Experience
- Professional Development
- Digital Resume & Portfolio

LEARN-2-EARN LABS TRAINING INSTITUTE

Head Office : F-4, First Floor, Anna Icon Complex, near Kargil Petrol Pump,
Sikandra-Bodla Road, Sikandra, Agra, Uttar Pradesh - 282007

Website : www.LearntoearnLabs.com

Call : 91-9548868337 / +91-9837705705

Insert Operations in MongoDB

- In MongoDB, insert operations refer to the process of adding new documents (also known as records or rows in traditional databases) to a collection.
- MongoDB is a NoSQL database that stores data in a flexible, JSON-like format called BSON (Binary JSON).

Before inserting anything, you need to have idea about the following terms:

- **Documents:** MongoDB stores data in collections, which are analogous to tables in relational databases. Each collection contains documents, which are similar to rows in relational databases but are JSON-like objects. Documents consist of key-value pairs, where keys are strings and values can be various data types, including other documents or objects or arrays.
- **Insertion:** To insert a document into a MongoDB collection, you typically use the `insertOne()` or `insertMany()` method provided by the MongoDB driver. The `insertOne()` method inserts a single document, while `insertMany()` can insert multiple documents at once.
- **Document Structure:** Before inserting documents into a collection, it's essential to define the structure of the documents. While MongoDB is schema-less in nature, meaning each document in a collection can have a different structure, having a consistent structure across documents can make querying and managing data more manageable.
- **Document IDs:** Each document in a MongoDB collection has a unique identifier called an ObjectId by default. This identifier is automatically generated by MongoDB and ensures that each document can be uniquely identified within the collection.
- **Atomicity:** MongoDB provides atomic operations at the document level. This means that when you perform an insert operation, it either fully succeeds or fails. There's no partially completed insert operation that leaves the database in an inconsistent state.
- **Indexes:** MongoDB allows you to define indexes on fields within documents. Indexes can improve query performance by enabling MongoDB to quickly locate documents based on indexed fields.
- **Write Concern:** MongoDB allows you to specify the level of acknowledgment required from the database when performing write operations. Write concern

options include acknowledging the write operation from the primary server, a subset of replica set members, or all replica set members.

- **Validation:** MongoDB provides validation rules that you can apply to collections to enforce data integrity. Validation rules can include enforcing the presence of certain fields, data types, or even custom validation logic using JavaScript functions.

Overall, MongoDB's approach to insert operations provides flexibility, scalability, and performance for handling various types of data in modern applications.

Various ways to perform insert operations in MongoDB

1. Insert a Single Document

To insert a single document into a collection, you can use the `insertOne()` method. This method takes an object representing the document to be inserted as its parameter.

```
db.collectionName.insertOne({ key: "value", key2: "value2" });
```

Example :

```
db.personal_Details.insertOne({id:45,name:"garima singh", city:"jaipur"})
```

If you want to see the inserted document then use command,

```
db.collectionName.find()
```

Example:

```
db.personal_Details.find()
```

If you want to insert an array as a key_value then use command

```
db.collectionName.insertOne({ key: "value", key2: "value2" , key3:["value_x",  
"value_y", "value_z"]});
```

Example:

```
db.personal_Details.insertOne({id:32, name:"aman bansal", city:"agra",  
skills:["java full stack", "mern stack", "DevOps", "UI/Ux"]})
```

If you want to insert an object as a key_value then use command

```
db.collectionName.insertOne({ key: "value", key2: "value2" ,  
key3:{key_x:"value_x", key_y:"value_y", key_z:"value_z"}});
```

Example:

```
db.personal_Details.insertOne({id:49,name:"pranjal",status:{frontend:"perfect",backend:"perfect",uiux:"excellent"}})
```

2. Insert Multiple Documents

To insert multiple documents into a collection, you can use the `insertMany()` method. This method takes an array of objects, with each object representing a document to be inserted.

```
db.collectionName.insertMany([
  { key: "value1", key2: "value2" },
  { key: "value3", key2: "value4" },
  { key: "value5", key2: "value6" } ]);
```

Example:

```
db.personal_Details.insertMany([
  {id:32, name:"tushar gupta", city:"Ghaziabad"},
  {id:47, name:"stuti singh", city:"Lucknow"},
  {id:54,name:"nikhil garg", city:"Agra"},
  {id:23,name:"neha garg", city:"mathura" } ]);
```

If you want to see the inserted document then use same command again,

```
db.collectionName.find()
```

Example:

```
db.personal_Details.find()
```

if you want to see the specific inserted document on the basis of respective key & value then use the command

```
db.collectionName.find({key:"value"})
```

Example:

```
db.personal_Details.find({id:54});
or db.personal_Details.find({name:"garima singh"});
or db.personal_Details.find({id:23, name:"neha garg"});
```

Inside the query object, if you specify the wrong key field and value pair then you will get nothing

3. Bulk Write Operations: MongoDB allows for bulk write operations, which include multiple insert operations executed at once.

```
db.collection.bulkWrite([
  { insertOne: { document: { ... } } },
  { insertOne: { document: { ... } } },
  // more operations
]);
```

Example:

```
db.personalDetails.bulkWrite([
  ... {insertOne:{document:{id:23,name:"priya"}}},
  ... {insertOne:{document:{id:17,name:"rishabh gupta"}}},
  ... {insertOne:{document:{id:19,name:"kanika sharma", city:"delhi"}}});
```

4. Insert with Explicit _id

By default, MongoDB automatically generates an _id field (primary_key/unique_id) for each document if it's not provided. However, you can specify the _id field explicitly for the document being inserted.

```
db.collectionName.insertOne({ _id: ObjectId("unique_id"), key: "value", key2:
"value2" });
```

Example:

```
db.personalDetails.insertMany([
  { _id: 1, name: "Neha Rathore", age: 23 },
  { _id: 2, name: "Rajat Sharma", age: 22 },
  { _id: 3, name: "Faisal Iqbal", age: 22 }  ]);
```

5. Insert with Validation

You can enable document validation during the insert operation to enforce a schema on the inserted documents. Document validation ensures that all documents meet certain criteria before they are inserted into the collection.

Example

```
db.createCollection("collectionName", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["id","name","city"],
      properties: {
        id: {
          bsonType: "number",
          description: "must be a number"
        },
        name: {
          bsonType: "string",
          description: "must be a string"  },
```

```
        city: {
          bsonType: "string",
          description: "must be a string"
        }
      }
    },
    validationAction: 'error'
  });
```

Where,

- **db.createCollection("collectionName"):** This part creates a new collection in the MongoDB database with the specified name.
- **validator:** This is an option that allows you to specify validation rules for the documents in the collection.
- **\$jsonSchema:** This specifies that the validation rules are defined using JSON Schema.
- **bsonType:** This specifies the BSON data type expected for each field.
- **required:** This specifies which fields are required in each document.
- **properties:** This object defines the properties (fields) of the documents and their validation rules.
- **description:** This provides a description for each property, explaining what type of data it should contain.
- **validationAction: 'error':** This specifies what action MongoDB should take when a document violates the validation rules. In this case, it will throw an error.

So, the above provided code creates a collection named "collectionName" where each document must contain fields "id," "name," and "city." The "id" field must be a number, while "name" and "city" must be strings. If a document fails to meet these requirements, MongoDB will throw an error.

Options available for insert operations

In MongoDB, there are several options available for insert operations that provide flexibility and control over how documents are inserted into a collection. These options allow you to customize the behavior of insert operations according to your specific requirements.

Here are some of the common options available for insert operations in MongoDB:

- **writeConcern:** This option specifies the level of acknowledgment requested from MongoDB for write operations. It allows you to control the durability of your write operations by specifying the write concern level. The possible values include w (write concern), wtimeout (write concern timeout), and j (journal acknowledgment).

For example:

```
db.collection.insertOne(document, { writeConcern: { w: "majority", wtimeout: 5000 } });
```

- **bypassDocumentValidation:** If set to true, this option allows the write operation to bypass document validation rules defined on the collection. It can be useful when you need to insert documents that do not satisfy the validation rules temporarily. For example:

```
db.collection.insertOne(document, { bypassDocumentValidation: true });
```

- **ordered (for insertMany()):** This option specifies whether the insert operation should be performed in order or in parallel for multiple documents inserted using insertMany(). If set to true, the operation is performed in order, and it stops on the first error encountered. If set to false, the operation is performed in parallel, and errors are reported after all documents are processed. Default is true. For example:

```
db.collection.insertMany(documents, { ordered: true });
```

- **session:** This option allows you to associate the insert operation with a specific session. Sessions are used for transactions in MongoDB, and associating an operation with a session ensures that it is part of the same transaction. For example:

```
db.collection.insertOne(document, { session: session });
```

These options provide control and flexibility when performing insert operations in MongoDB, allowing you to tailor the behavior of insert operations according to your application's requirements. Always ensure that you understand the implications of each option before using it in your code.

Insert with Write Concern

- Write concern determines the level of acknowledgment requested from MongoDB for write operations. You can specify the write concern for the insert operation.
- Write concern describes the level of acknowledgment requested from MongoDB for write operations to a standalone mongod, replica sets, or sharded clusters. In sharded clusters, mongos instances will pass the write concern on to the shards.

Write concern can include the following fields:

`{ w: <value>, j: <boolean>, wtimeout: <number> }`

w option

- the **w option** to request acknowledgment that the write operation has propagated to a specified number of mongod instances or to mongod instances with specified tags.

For example:

- **w: 1**

Requests acknowledgment that the write operation has propagated to the standalone mongod or the primary in a replica set. Data can be rolled back if the primary steps down before the write operations have replicated to any of the secondaries.

- **w: 0**

Requests no acknowledgment of the write operation. However, w: 0 may return information about socket exceptions and networking errors to the application. Data can be rolled back if the primary steps down before the write operations have replicated to any of the secondaries.

j option

- the **j option** to request acknowledgment that the write operation has been written to the on-disk journal. By default the value is j:false.
- j:true means, MongoDB returns only after the requested number of members, including the primary, have written to the journal. Previously j: true write concern in a replica set only requires the primary to write to the journal, regardless of the w: <value> write concern.

wtimeout

- This option specifies a time limit, in milliseconds, to prevent write operations from blocking indefinitely for the write concern. wtimeout is only applicable for w values greater than 1.
- wtimeout causes write operations to return with an error after the specified limit, even if the required write concern will eventually succeed. When these write operations return, MongoDB does not undo successful data modifications performed before the write concern exceeded the wtimeout time limit.
- If you do not specify the wtimeout option and the level of write concern is unachievable, the write operation will block indefinitely. Specifying a wtimeout value of 0 is equivalent to a write concern without the wtimeout option.

```
db.collectionName.insertOne({ key: "value", key2: "value2" }, { writeConcern: { w: 0, j: true } });
```

Example:

```
db.personalDetails.insertOne({id:23, name:"riya sharma"}, {writeConcern:{w:0,j:true}})
```

Insert with bypassDocumentValidation

If you have created a collection in a MongoDB database with validation rules applied to the documents inserted into that collection using the following code snippet:

```
db.createCollection('data',{
  validator:{
    $jsonSchema:{
      bsonType:"object",
      required:["id","name"],
      properties:{
        id:{
          bsonType:"number",
          description:"id should be a number"},
        name:{
          bsonType:"string",
          description:"name must be a string"}
      }
    },validationAction:'error'});
```


& if you want to insert the below document inside the created collection with validation rules:

```
db.data.insertOne({id:"hello",name:"rashmi"})
```

then you will get **MongoServerError: Document failed validation**

so if it is required to insert the above document then you need to bypass the document validation using the below code snippet or command:

```
db.data.insertOne({id:"hello",name:"rashmi"}, {  
... bypassDocumentValidation:true})
```

Ordered Insert and Unordered Insert

In MongoDB, when inserting multiple documents into a collection, you have the option to perform ordered inserts or unordered inserts. Let's understand each:

- **Ordered Inserts**

- In ordered inserts, MongoDB processes the documents in the order they are specified in the bulk operation.
- If an error occurs during the insert of any document, MongoDB will halt further processing and return an error message. It will not continue with the insertion of subsequent documents.
- The error message will indicate which document caused the error, allowing you to identify and handle the issue.
- Ordered inserts are the default behavior in MongoDB.

Example:

```
db.collection.insertMany([  
  { _id: 1, name: "Neha Rathore" },  
  { _id: 2, name: "Shivani Sharma" },  
  { _id: 2, name: "Sankalp Saxena" } here halt further processing and  
                                     return an error message  
  { _id: 3, name: "Nishant Kumar" }  
  { _id: 4, name: "Tushar Gupta" }  
],  
 { ordered: true });
```

Inserted count:2 & err:index:2

- **Unordered Inserts**

- In unordered inserts, MongoDB attempts to process all documents in the bulk operation regardless of any errors encountered.
- If an error occurs during the insertion of any document, MongoDB will continue inserting subsequent documents and return a single error message containing information about all failed documents.
- Unordered inserts can be faster than ordered inserts since MongoDB doesn't have to halt the operation on encountering an error.

Example:

```
db.collection.insertMany([
  { _id: 1, name: "Neha Rathore" },
  { _id: 2, name: "Shivani Sharma" },
  { _id: 2, name: "Sankalp Saxena" } //return an error message but here
                                     further processing not halted
  { _id: 3, name: "Nishant Kumar" }
  { _id: 4, name: "Tushar Gupta" }
],
{ ordered: false });
```

Inserted count:4 & err:index:2

Insert with Session

In MongoDB, a session represents a logical unit of work. Sessions are useful for transactions and for ensuring consistency and isolation across multiple operations. Sessions were introduced in MongoDB 4.0 to support multi-document transactions. When performing insert operations within a session in MongoDB, you typically start a session, perform your insert operations, and then commit the session to make the changes permanent or abort the session to discard the changes.

Join us for Industry Based & Job Oriented Training Programs

Top Edge Training Programs



**MERN / MEAN Stack
Development**



**Java Full Stack
Development**



**Data Science
& Machine Learning**



**Data Analytics &
Business Intelligence**



**Linux, Cloud
& DevOps**



**Digital Marketing &
Business Development**

Amenities

- Digital Notes
- Learning References
- Live Training Sessions
- Live Project Training
- Interview Preparation
- Live Session Recordings for Revision
- Job Referrals & Recommendations
- Project Work Experience
- Professional Development
- Digital Resume & Portfolio

LEARN-2-EARN LABS TRAINING INSTITUTE

Head Office : F-4, First Floor, Anna Icon Complex, near Kargil Petrol Pump,
Sikandra-Bodla Road, Sikandra, Agra, Uttar Pradesh - 282007

Website : www.LearntoearnLabs.com

Call : 91-9548868337 / +91-9837705705

Read operations in MongoDB

Read operations in MongoDB are used to retrieve data from the database. MongoDB provides various methods and commands for reading data.

Here are some of the common read operations in MongoDB:

- **find():** The find() method is used to retrieve documents from a collection that match a specified query criteria. It returns a cursor to the selected documents.

Example:

```
db.collection.find();  
Or db.collection.find({ key: value });
```

- **findOne():** The findOne() method retrieves a single document from the collection that matches the specified query criteria. It returns the first matching document.

Example:

```
db.collection.findOne(); //retrieves first document of collection  
Or db.collection.findOne({key:value});  
//retrieves the first document that matches the specified query criteria
```

- **count():** The count() method returns the count of documents that match the specified query criteria.

Example:

```
db.collection.count(); //returns the total number of documents  
Or db.collection.count({key:value});  
// returns the count of documents that match the specified query criteria
```

Now, Collection.count() is deprecated, so you can use countDocuments()

```
db.collection.countDocuments();  
//returns the total number of documents  
Or db.collection.countDocuments({key:value});  
//returns the count of documents that match the specified query criteria
```

- **distinct():** The distinct() method returns an array of distinct values for a specified field within the collection.

Example:

```
db.collection.distinct('key');
```

- **aggregate():** The aggregate() method allows you to perform advanced aggregation operations such as grouping, sorting, and filtering documents.

Example:

```
db.collection.aggregate([
  { $match: { field: value } },
  { $group: { _id: "$field", count: { $sum: 1 } } }
]);
```

Suppose, we have a collection: orders

```
db.orders.insertMany([
  { "customer": "neha", "product": "watches", "quantity": 5},
  { "customer": "sandeep", "product": "glasses", "quantity": 8},
  { "customer": "tushar", "product": "laptops", "quantity": 12},
  { "customer": "megha", "product": "monitors", "quantity": 6},
  { "customer": "garima", "product": "digital tablets", "quantity": 10},
  { "customer": "rajat", "product": "cameras", "quantity": 7},
  { "customer": "sankalp", "product": "mobiles", "quantity": 15}])
```

So, you need to use the following code snippet for grouping, sorting and filtering:

```
db.orders.aggregate([
  // Grouping by customer and calculating total quantity
  {
    $group: {
      _id: "$customer", // Group by customer
      totalQuantity: { $sum: "$quantity" } // Calculate total quantity for each
customer
    }
  },
  // Sorting by total quantity in descending order
  {
    $sort: { totalQuantity: -1 } // Sort by total quantity in descending order
  },
  // Filtering customers who have ordered more than 10 products
  {
    $match: { totalQuantity: { $gt: 10 } } // Filter out customers with total quantity
greater than 10
  }
])
```

- **findAndModify():** The findAndModify() method is used to atomically find and modify a single document. It can perform operations like update, delete, or return the modified document.

The findAndModify method is used to query and modify documents atomically. It's a single operation that finds a document based on certain criteria, updates it according to specified modifications, and returns either the original document as it was before the modifications or the modified document.

Example:

```
db.collection.findAndModify({  
  query: { field: value },  
  update: { $set: { newField: newValue } }  
});
```

To modify the complete document on the basis of specified query criteria inside the collection:

```
db.orders.findAndModify({ query: { customer: "sankalp" }, update:{ name:  
"rishabh" } })
```

To add some properties to the existing document on the basis of specified query criteria inside the collection:

```
db.orders.findAndModify({ query: { customer: "garima" }, update: { $set: {  
gender: "female" } } })
```

To modify the existing property (key_value) of the existing document on the basis of specified query criteria

```
db.orders.findAndModify({ query: { customer: "garima" }, update: { $set: {  
customer: "rajni verma" } } })
```

You can also explore findAndModify() on the basis of below syntax:

```
db.collection.findAndModify({
  query: <document>,    // Specifies the selection criteria for the modification
  sort: <document>,     // Specifies the sort order for the operation
  remove: <boolean>,    // Optional. If true, removes the document. Default is false.
  update: <document>,   // Optional. Specifies the modifications to apply.
  new: <boolean>,       // Optional. If true, returns the modified document. Default is
                        // false.
  fields: <document>,   // Optional. Specifies the fields to return. Default is all fields.
  upsert: <boolean>,    // Optional. If true, creates a new document if no document
                        // matches the query. Default is false.
  bypassDocumentValidation: <boolean>, // Optional. If true, bypasses document
                        // validation. Default is false.
  writeConcern: <document>, // Optional. Specifies the write concern.
  collation: <document>, // Optional. Specifies the collation settings for the operation.
  arrayFilters: [ <filterdocument1>, ... ], // Optional. Specifies array filters for an update
                                                // operation.
})
```

LABS

Importing Json files

Create two json files in your pc/laptop.

orders.json

```
[
  { "customer": "neha", "product": "watches", "quantity": 5},
  { "customer": "sandeep", "product": "glasses", "quantity": 8},
  { "customer": "tushar", "product": "laptops", "quantity": 12},
  { "customer": "megha", "product": "monitors", "quantity": 6},
  { "customer": "garima", "product": "digital tablets", "quantity": 10},
  { "customer": "rajat", "product": "cameras", "quantity": 7},
  { "customer": "sankalp", "product": "mobiles", "quantity": 15}
]
```

todos.json

get the data using link : <https://jsonplaceholder.typicode.com/todos>

Now open command prompt and run the following commands

```
mongoimport --file "E:\json data\todos.json" --db importdb --collection todosCollection
--jsonArray
```

and

```
mongoimport --file "E:\json data\orders.json" --db importdb --collection ordersCollection
--jsonArray
```

where "E:/json data" is the folder/directory where the files are located.

Also make sure that the following path should be added to your system environment variable:

C:\Program Files\MongoDB\Tools\100\bin

Exporting MongoDB collection

Exporting a MongoDB collection involves extracting data from a specific collection within a MongoDB database and saving it in a format that can be used for backup, migration, analysis, or sharing with others.

MongoDB provides several methods for exporting collections, including using built-in tools like mongoexport command-line tool or writing custom scripts.

Here's a detailed explanation of how to export a MongoDB collection:

- **Using mongoexport Tool:** MongoDB provides a command-line tool called mongoexport, which allows you to export data from a collection to a JSON, CSV, or other supported format. The basic syntax of the mongoexport command is:
`mongoexport --uri <connection_uri> --collection <collection_name> --out <output_file_path>`

For Example:

```
mongoexport --uri "mongodb://localhost:27017/importdb" --collection  
ordersCollection --out "e:\json data\exportOrders.json"
```

or you can use,

```
mongoexport --out "E:\json data\exportTodos.json" --db importdb --collection  
todosCollection
```

Also make sure that the following path should be added to your system environment variable:

C:\Program Files\MongoDB\Tools\100\bin

- **Using MongoDB Drivers:** If you're developing an application, you can use MongoDB drivers for your programming language to connect to the database, retrieve data from the collection, and save it to a file in the desired format programmatically.
- **Exporting to Other Formats:** While mongoexport primarily exports data to JSON or CSV formats, you can further process the exported data to convert it into other formats using scripting languages or tools specific to your use case.

Considerations during exporting MongoDB collection

- **Authentication:** Ensure that you have the necessary permissions to access the MongoDB collection you intend to export.
- **Data Volume:** Depending on the size of your collection and available system resources, exporting large collections may require additional considerations to avoid performance issues.
- **Data Consistency:** MongoDB collections can be dynamic, so ensure data consistency during the export process, especially if the collection schema has changed over time.

MongoDB Operators

- In MongoDB, operators are used to perform various operations on data such as comparison, logical operations, array operations, etc.
- They are essential for querying and manipulating data within MongoDB collections.
- MongoDB provides a rich set of operators to perform different types of operations.

Here's an overview of some common types of operators in MongoDB:

1. Comparison Operators

- **\$eq:** Matches values that are equal to a specified value.
`db.collection.find({ field: { $eq: value } })`
- **\$ne:** Matches values that are not equal to a specified value.
`db.collection.find({ field: { $ne: value } })`
- **\$gt:** Matches values that are greater than a specified value.
`db.collection.find({ field: { $gt: value } })`
- **\$gte:** Matches values that are greater than or equal to a specified value.
`db.collection.find({ field: { $gte: value } })`
- **\$lt:** Matches values that are less than a specified value.
`db.collection.find({ field: { $lt: value } })`

- **\$lte:** Matches values that are less than or equal to a specified value.
`db.collection.find({ field: { $lte: value } })`
- **\$in:** Matches any of the values specified in an array.
`db.collection.find({ field: { $in: [value1, value2, ...] } })`
- **\$nin:** Matches none of the values specified in an array.
`db.collection.find({ field: { $nin: [value1, value2, ...] } })`
- **\$exists:** Matches documents that have the specified field.
`db.collection.find({ field: { $exists: true } })`
- **\$type:** Matches documents where the value of a field is of the specified BSON type.
`db.collection.find({ field: { $type: type } })`
- **\$text:** Performs a text search on the content of the fields indexed with a text index.
`db.collection.find({ $text: { $search: "search string" } })`

2. Logical Operators

- **\$and:** Joins query clauses with a logical AND and returns all documents that match the conditions.
`db.collection.find({ $and: [{ condition1 }, { condition2 } ] })`
- **\$or:** Joins query clauses with a logical OR and returns all documents that match at least one of the conditions.
`db.collection.find({ $or: [{ condition1 }, { condition2 } ] })`
- **\$not:** Inverts the effect of a query expression and returns documents that do not match the query expression.
`db.collection.find({ field: { $not: { $eq: value } } })`
- **\$nor:** Joins query clauses with a logical NOR and returns all documents that fail to match both conditions.
`db.collection.find({ $nor: [{ condition1 }, { condition2 } ] })`

3. Array Operators

- **\$elemMatch:** Matches documents that contain an array field with at least one element that matches all the specified query criteria.
`db.collection.find({ field: { $elemMatch: { condition1, condition2 } } })`
- **\$size:** Matches arrays with a specified number of elements.
`db.collection.find({ field: { $size: sizeValue } })`
- **\$all:** Matches arrays that contain all elements specified in the query.
`db.collection.find({ field: { $all: [element1, element2, ...] } })`
- **\$arrayElemAt:** Returns the element at the specified array index.
`db.collection.aggregate([
 { $project: { firstElement: { $arrayElemAt: ["$arrayField", 0] } } }
])`
- **\$filter:** Filters the elements of an array based on a condition.
`db.collection.aggregate([
 { $project: { filteredArray: { $filter: { input: "$arrayField", as: "item", cond: {
 $gte: ["$$item", 10] } } } } }
])`
- **\$arrayToObject:** Converts an array into a single document.
- **\$concatArrays:** Concatenates arrays to return the concatenated array.
- **\$isArray:** Determines whether the operand is an array.
- **\$map:** Applies an expression to each element in an array and returns an array with the applied results.
- **\$reduce:** Applies an expression to each element in an array and combines them into a single value.
- **\$slice:** Returns a subset of an array.
- **\$zip:** Aggregates multiple arrays together element-by-element.

- **\$reverseArray:** Returns an array with the elements in reverse order.

4. Array Update Operators

- **\$addToSet:** Adds elements to an array only if they are not already present in it.

```
db.collection.updateOne({ _id: ObjectId("document_id") }, { $addToSet: { field: value } })
```
- **\$push:** Adds an element to an array field.

```
db.collection.updateOne({ _id: ObjectId("document_id") }, { $push: { field: value } })
```
- **\$pull:** Removes all occurrences of a value from an array field.

```
db.collection.updateOne({ _id: ObjectId("document_id") }, { $pull: { field: value } })
```
- **\$pop:** Removes the first or last element of an array.

```
db.collection.updateOne({ _id: ObjectId("document_id") }, { $pop: { field: 1 } })
```

5. Evaluation Operators (Optional)

- **\$expr:** Allows the use of aggregation expressions within the query language.

```
db.collection.find({ $expr: { $gt: ["$field1", "$field2"] } })
```
- **\$regex:** Matches documents that have a field matching a regular expression.

```
db.collection.find({ field: { $regex: /pattern/ } })
```

6. Text Search Operator

- **\$text:** Performs a text search on the content of indexed fields.

```
db.collection.find({ $text: { $search: "search string" } })
```
- **\$meta:** Returns metadata related to text search scoring.

7. Type Conversion Operators

- **\$toInt:** Converts a value to an integer.

```
db.collection.aggregate([  
  { $project: { integerField: { $toInt: "$stringField" } } }  
])
```
- **\$toDouble:** Converts a value to a double.

```
db.collection.aggregate([  
  { $project: { doubleField: { $toDouble: "$stringField" } } }  
])
```
- **\$convert:** Converts a value to a specified BSON type.
- **\$toBool:** Converts a value to a boolean.
- **\$toDate:** Converts a value to a date.
- **\$toDecimal:** Converts a value to a decimal.
- **\$toLong:** Converts a value to a long.
- **\$toObjectId:** Converts a value to an ObjectId.
- **\$toString:** Converts a value to a string.

8. Aggregation Operators

- **\$match:** Filters documents by a specific condition before passing them to the next stage in an aggregation pipeline.

```
db.collection.aggregate([  
  { $match: { field: value } }  
])
```
- **\$group:** Groups documents by a specified expression and applies accumulator expressions to the grouped documents.


```
db.collection.aggregate([
  { $group: { _id: "$field", total: { $sum: 1 } } }
])
```

- **\$project:** Reshapes documents by including, excluding, or renaming fields and by adding computed fields.

```
db.collection.aggregate([
  { $project: { newField: "$existingField", _id: 0 } }
])
```

- **\$sort:** Orders the documents returned by a query.
- **\$limit:** Limits the number of documents returned by a query.
- **\$skip:** Skips a specified number of documents in a query and returns the rest.

9. Conditional Operators

- **\$cond:** Evaluates a boolean expression and returns one of two specified expressions based on the result.
- **\$switch:** Evaluates a series of case expressions and returns the result of the first case that matches, similar to how a switch statement works.
- **\$ifNull:** Returns the value of the first expression if it is not null; otherwise, returns the value of the second expression.

10. Set Operators

- **\$setUnion:** Returns the union of two or more sets.

```
db.collection.aggregate([
  { $project: { unionSet: { $setUnion: ["$set1", "$set2"] } } }
])
```

- **\$setDifference:** Returns the difference between two sets.

```
db.collection.aggregate([
  { $project: { differenceSet: { $setDifference: ["$set1", "$set2"] } } }
```

```
])
```

- **\$setIntersection:** Returns elements that appear in all input sets.
- **\$setIsSubset:** Returns true if all elements of the first set appear in the second set.
- **\$setEquals:** Returns true if the input sets have the same distinct elements.

11.Array Expression Operators

- **\$size:** Returns the number of elements in an array.

```
db.collection.aggregate([  
  { $project: { arraySize: { $size: "$arrayField" } } }  
])
```
- **\$arrayToObject:** Converts an array into a document.

```
db.collection.aggregate([  
  { $project: { arrayAsObject: { $arrayToObject: [["key1", "value1"], ["key2",  
"value2"]] } } }  
])
```

12.Geospatial Aggregation Operators (Optional)

- **\$geoNear:** Returns an ordered stream of documents based on the proximity to a specified geospatial point.
- **\$geoWithin:** Selects documents with geospatial data that exists entirely within a specified shape.
- **\$geoIntersects:** Selects documents with geospatial data that intersects with a specified shape.

13.Date Operators

- **\$dateFromString:** Converts a string to a date.

- **\$dateToString:** Converts a date to a string.
- **\$dateToParts:** Returns the parts of a date as a document.
- **\$dateFromParts:** Constructs a date from separate parts.
- **\$dayOfWeek:** Returns the day of the week for a date as a number between 1 (Sunday) and 7 (Saturday).
- **\$month:** Returns the month for a date as a number between 1 (January) and 12 (December).
- **\$year:** Returns the year for a date.
- **\$dayOfMonth:** Returns the day of the month from a date.
- **\$hour:** Returns the hour from a date.
- **\$minute:** Returns the minute from a date.
- **\$second:** Returns the second from a date.
- **\$millisecond:** Returns the millisecond from a date.

14.String Operators

- **\$strcasecmp:** Compares two strings case-insensitively and returns:
 - 0 if the strings are equal.
 - 1 if the first string is greater than the second.
 - -1 if the first string is less than the second.
- **\$toLower:** Converts a string to lowercase.
- **\$toUpper:** Converts a string to uppercase.
- **\$split:** Splits a string into substrings based on a delimiter and returns an array.

- **\$strLenBytes:** Returns the number of bytes in a string.
- **\$strLenCP:** Returns the number of UTF-8 code points in a string.

15.Variable Operators

- **\$let:** Binds variables for use in a sub-expression and returns the result of the sub-expression.
- **\$mergeObjects:** Merges multiple documents into a single document.
- **\$objectToArray:** Converts a document to an array of key-value pairs.
- **\$literal:** Returns a value without parsing.
- **\$rand:** Generates a random floating-point number between 0 and 1.
- **\$randInt:** Generates a random integer within a specified range.

16.Type Operators

- **\$type:** Returns the BSON type of a field.
- **\$isArray:** Determines whether a value is an array.
- **\$isNumber:** Determines whether a value is a number.
- **\$isString:** Determines whether a value is a string.

17.Comments Operator

- **\$comment:** Adds a comment to a query. This is useful for documenting and debugging queries.

18. Graph Lookup Operator (Optional)

- **\$graphLookup:** Performs recursive search on a collection, constructing a tree structure, and returning the results.

19. Redaction Operator

- **\$redact:** Controls the access to data in documents during aggregation.

20. Unary Operators

- **\$abs:** Returns the absolute value of a number.
- **\$ceil:** Rounds a number up to the nearest integer.
- **\$floor:** Rounds a number down to the nearest integer.
- **\$ln:** Calculates the natural logarithm of a number.
- **\$log:** Calculates the logarithm of a number with a specified base.
- **\$sqrt:** Calculates the square root of a number.
- **\$trunc:** Truncates a number to an integer.

21. Update Operators

- **\$currentDate:** Sets the value of a field to the current date or time.
- **\$inc:** Increments the value of the field by a specified amount.
- **\$min:** Updates the value of the field to the specified value if the specified value is less than the current value.
- **\$max:** Updates the value of the field to the specified value if the specified value is greater than the current value.

- **\$mul:** Multiplies the value of the field by a specified amount.
- **\$rename:** Renames a field.
- **\$set:** Sets the value of a field in a document.
- **\$unset:** Removes the specified field from a document.

22.Arithmetic Operators

- **\$add:** Adds numbers together.
- **\$subtract:** Subtracts one number from another.
- **\$multiply:** Multiplies numbers together.
- **\$divide:** Divides one number by another.

23.Miscellaneous Operators (Optional)

- **\$accumulator:** Defines custom aggregation functions.
- **\$currentOp:** Returns information on current operations running on the server.
- **\$indexStats:** Returns statistics on index usage.

Join us for Industry Based & Job Oriented Training Programs

Top Edge Training Programs



**MERN / MEAN Stack
Development**



**Java Full Stack
Development**



**Data Science
& Machine Learning**



**Data Analytics &
Business Intelligence**



**Linux, Cloud
& DevOps**



**Digital Marketing &
Business Development**

Amenities

- Digital Notes
- Learning References
- Live Training Sessions
- Live Project Training
- Interview Preparation
- Live Session Recordings for Revision
- Job Referrals & Recommendations
- Project Work Experience
- Professional Development
- Digital Resume & Portfolio

LEARN-2-EARN LABS TRAINING INSTITUTE

Head Office : F-4, First Floor, Anna Icon Complex, near Kargil Petrol Pump,
Sikandra-Bodla Road, Sikandra, Agra, Uttar Pradesh - 282007

Website : www.LearntoearnLabs.com

Call : 91-9548868337 / +91-9837705705

Exercises : Operators in MongoDB

```
db.users.insertMany([
  { "_id": 1, "name": "Alisha", "age": 30, "status": "A" },
  { "_id": 2, "name": "Rohan", "age": 25, "status": "B" },
  { "_id": 3, "name": "Deepali", "age": 35, "status": "A" },
  { "_id": 4, "name": "Sameer", "age": 20, "status": "C" },
  { "_id": 5, "name": "Karan", "age": 40, "status": "B" }
]);
```

Comparison Operators

- **Equals (\$eq)** : Find users with age equal to 30.
`db.users.find({ age: { $eq: 30 } })`
- **Not Equals (\$ne)**: Find users with age not equal to 25.
`db.users.find({ age: { $ne: 25 } })`
- **Greater Than (\$gt)**: Find users with age greater than 30.
`db.users.find({ age: { $gt: 30 } })`
- **Greater Than or Equal To (\$gte)**: Find users with age greater than or equal to 25
`db.users.find({ age: { $gte: 25 } })`
- **Less Than (\$lt)**: Find users with age less than 35.
`db.users.find({ age: { $lt: 35 } })`
- **Less Than or Equal To (\$lte)**: Find users with age less than or equal to 40.
`db.users.find({ age: { $lte: 40 } })`
- **In (\$in)**: Find users with status "A" or "B".
`db.users.find({ status: { $in: ["A", "B"] } })`
- **Not In (\$nin)**: Find users with status other than "A" or "B".
`db.users.find({ status: { $nin: ["A", "B"] } })`
- **Exists (\$exists)**: Find users where the status field exists.
`db.users.find({ status: { $exists: true } })`
- **Type (\$type)**: Find users where the age field is of type number.
`db.users.find({ age: { $type: "number" } })`

Logical Operators

- **AND (\$and):** Joins query clauses with a logical AND and returns all documents that match the conditions.

```
db.users.find({ $and: [ { status: "A" }, { age: { $lt: 35 } } ] })
```

This query finds users where the status is "A" and the age is less than 35.

- **OR (\$or):** Joins query clauses with a logical OR and returns all documents that match at least one of the conditions.

```
db.users.find({ $or: [ { status: "A" }, { age: { $lt: 25 } } ] })
```

This query finds users where the status is "A" or the age is less than 25.

- **NOT (\$not):** Inverts the effect of a query expression and returns documents that do not match the query expression.

```
db.users.find({ age: { $not: { $gt: 30 } } })
```

This query finds users where the age is not greater than 30.

- **NOR (\$nor):** Joins query clauses with a logical NOR and returns all documents that fail to match both conditions.

```
db.users.find({ $nor: [ { status: "A" }, { age: { $lt: 25 } } ] })
```

LABS

let's take another collection

```
db.students.insertMany([
  {
    name: "Sameer",
    scores: [
      { type: "exam", score: 85 },
      { type: "quiz", score: 90 },
      { type: "homework", score: 88 }
    ],
    tags: ["math", "science", "history"]
  },
  {
    name: "Alisha",
    scores: [
      { type: "exam", score: 95 },
      { type: "quiz", score: 88 },
      { type: "homework", score: 92 }
    ],
    tags: ["math", "physics", "chemistry"]
  },
  {
    name: "Karan",
    scores: [
      { type: "exam", score: 78 },
      { type: "quiz", score: 85 },
      { type: "homework", score: 80 }
    ],
    tags: ["biology", "geography", "history"]
  }
])
```

Array Operators

- **\$elemMatch:** Matches documents that contain an array field with at least one element that matches all the specified query criteria.
`db.students.find({ scores: { $elemMatch: { type: "exam", score: { $gt: 90 } } } })`
- **\$size:** Matches arrays with a specified number of elements.
`db.students.find({ tags: { $size: 3 } })`
- **\$all:** Matches arrays that contain all elements specified in the query.
`db.students.find({ tags: { $all: ["math", "science", "history"] } })`
- **\$arrayElemAt:** Returns the element at the specified array index.
`db.students.aggregate([ { $project: { firstScore: { $arrayElemAt: ["$scores", 0] } } } ])`
- **\$filter:** Filters the elements of an array based on a condition.
`db.students.aggregate([ { $project: { highScores: { $filter: { input: "$scores", as: "score", cond: { $gte: ["$$score.score", 80] } } } } } ])`

Let's take another collection

```
db.students.insertMany([
  {
    _id: 1,
    name: "Sameer",
    hobbies: ["reading", "cooking"]
  },
  {
    _id: 2,
    name: "Alisha",
    hobbies: ["sports", "music"]
  },
  {
    _id: 3,
    name: "Sameer",
    hobbies: ["reading", "gardening"]
  }
])
```

Array Update Operators

- **\$addToSet:** Adds elements to an array only if they are not already present in it.
`db.students.updateOne({ _id: 1 }, { $addToSet: { hobbies: "drawing" } })`
After this operation, the document with `_id` equal to 1 will have the hobbies array ["reading", "cooking", "drawing"].
- **\$push:** Adds an element to an array field.
`db.students.updateOne({ _id: 2 }, { $push: { hobbies: "painting" } })`
After this operation, the document with `_id` equal to 2 will have the hobbies array ["sports", "music", "painting"].
- **\$pull:** Removes all occurrences of a value from an array field.
`db.students.updateOne({ _id: 3 }, { $pull: { hobbies: "reading" } })`
After this operation, the document with `_id` equal to 3 will have the hobbies array ["gardening"].
- **\$pop:** Removes the element of an array.
`db.students.updateOne({ _id: 1 }, { $pop: { hobbies: 1 } })`
use hobbies: 1 to remove the last element & hobbies: -1 to remove the first element

Let's take another collection

```
db.articles.insertMany([
  {
    title: "Introduction to MongoDB",
    content: "MongoDB is a popular NoSQL database."
  },
  {
    title: "MongoDB Text Search",
    content: "MongoDB provides powerful text search capabilities."
  },
  {
    title: "Using MongoDB with Node.js",
    content: "Node.js is a JavaScript runtime that can be used with MongoDB."
  }
])
```

Text Search Operator

```
db.articles.createIndex({ content: "text" })
```

Now, we can perform a text search using the \$text operator:

```
db.articles.find({ $text: { $search: "MongoDB text search" } })
```

This query will return documents from the articles collection where the content field contains the words "MongoDB", "text", or "search".

we can also use the \$meta operator to include the text search score in the results:

```
db.articles.find( { $text: { $search: "MongoDB text search" } }, { score: { $meta: "textScore" } } ).sort({ score: { $meta: "textScore" } })
```

Let's take another collection

```
db.items.insertMany([
  {
    name: "Item 1",
    quantity: "10",
    price: "20.5",
    isActive: "true",
    createdAt: "2022-03-01T10:00:00Z"
  },
  {
    name: "Item 2",
    quantity: "20",
    price: "15.75",
    isActive: "false",
    createdAt: "2022-03-02T11:00:00Z"
  },
  {
    name: "Item 3",
    quantity: "30",
    price: "25.0",
    isActive: "true",
    createdAt: "2022-03-03T12:00:00Z"
  }
])
```

Type Conversion Operators

- **\$toInt:** Convert the "quantity" field from string to integer.
`db.items.aggregate([ { $project: { quantityInt: { $toInt: "$quantity" } } } ])`
- **\$toDouble:** Convert the "price" field from string to double.
`db.items.aggregate([ { $project: { priceDouble: { $toDouble: "$price" } } } ])`
- **\$toBool:** Convert the "isActive" field from string to boolean.
`db.items.aggregate([ { $project: { isActiveBool: { $toBool: "$isActive" } } } ])`
- **\$toDate:** Convert the "createdAt" field from string to date.
`db.items.aggregate([ { $project: { createdAtDate: { $toDate: "$createdAt" } } } ])`
- **\$toString:** Convert the "quantity" field from integer to string.
`db.items.aggregate([ { $project: { quantityString: { $toString: "$quantity" } } } ])`

Let's take another collection

```
db.sales.insertMany([
  { _id: 1, item: "A", quantity: 10, price: 20 },
  { _id: 2, item: "B", quantity: 20, price: 15 },
  { _id: 3, item: "A", quantity: 15, price: 25 },
  { _id: 4, item: "C", quantity: 5, price: 30 },
  { _id: 5, item: "B", quantity: 10, price: 18 }
])
```

Aggregation Operators

- **\$match:** Filter documents by a specific condition before passing them to the next stage in an aggregation pipeline.
`db.sales.aggregate([ { $match: { item: "A" } } ])`
- **\$group:** Group documents by a specified expression and apply accumulator expressions to the grouped documents.
`db.sales.aggregate([ { $group: { _id: "$item", totalQuantity: { $sum: "$quantity" } } } ])`

- **\$project:** Reshape documents by including, excluding, or renaming fields and by adding computed fields.
`db.sales.aggregate([ { $project: { item: 1, totalAmount: { $multiply: ["$quantity", "$price"] } } } ])`
- **\$sort:** Order the documents returned by a query.
`db.sales.aggregate([ { $sort: { price: -1 } } ])` //descending order
`db.sales.aggregate([ { $sort: { price: 1 } } ])` //ascending order
- **\$limit:** Limit the number of documents returned by a query.
`db.sales.aggregate([ { $limit: 2 } ])`
- **\$skip:** Skip a specified number of documents.
`db.sales.aggregate([ { $skip: 2 } ])`

Let's take another collection

```
db.products.insertMany([
  { _id: 1, name: "Product 1", quantity: 15, price: 20 },
  { _id: 2, name: "Product 2", quantity: 5, price: 30 },
  { _id: 3, name: "Product 3", quantity: 25, price: 15 },
  { _id: 4, name: "Product 4", quantity: 10, price: 25 },
  { _id: 5, name: "Product 5", quantity: null, price: 10 }
])
```

Conditional Operators

- **\$cond:** Evaluates a boolean expression and returns one of two specified expressions based on the result.
`db.products.aggregate([ { $project: { status: { $cond: { if: { $gte: ["$quantity", 10] }, then: "In stock", else: "Out of stock" } } } } ])`
- **\$switch:** Evaluates a series of case expressions and returns the result of the first case that matches.
`db.products.aggregate([ { $project: { discount: { $switch: { branches: [ { case: { $gte: ["$quantity", 20] }, then: 0.2 }, { case: { $gte: ["$quantity", 10] }, then: 0.1 }, { case: { $lt: ["$quantity", 10] }, then: 0 } ], default: 0 } } } } ])`

- **\$ifNull:** Returns the value of the first expression if it is not null; otherwise, returns the result of the second expression.

```
db.products.aggregate([ { $project: { result: { $ifNull: ["$quantity", "$price"] } } } ] )
```

or

```
db.products.aggregate([ { $project: { result: { $ifNull: ["$quantity", "Value is null"] } } } ] )
```

Let's take another collection

```
db.sets.insertMany([
    { _id: 1, set1: [1, 2, 3, 4], set2: [3, 4, 5, 6] },
    { _id: 2, set1: [2, 4, 6, 8], set2: [1, 3, 5, 7] },
    { _id: 3, set1: [1, 3, 5, 7], set2: [2, 4, 6, 8] }
])
```

Set Operators

- **\$setUnion:** Returns the union of two or more sets.

```
db.sets.aggregate([ { $project: { unionSet: { $setUnion: ["$set1", "$set2"] } } } ] )
```

- **\$setDifference:** Returns the difference between two sets.

```
db.sets.aggregate([ { $project: { differenceSet: { $setDifference: ["$set1", "$set2"] } } } ] )
```

- **\$setIntersection:** Returns elements that appear in all input sets.

```
db.sets.aggregate([ { $project: { intersectionSet: { $setIntersection: ["$set1", "$set2"] } } } ] )
```

- **\$setIsSubset:** Returns true if all elements of the first set appear in the second set.

```
db.sets.aggregate([ { $project: { isSubset: { $setIsSubset: ["$set1", "$set2"] } } } ] )
```

Let's take another collection

```
db.students.insertMany([
    { _id: 1, name: "Megha", subjects: ["Math", "Science", "History"] },
    { _id: 2, name: "Tushar", subjects: ["English", "Geography"] },
    { _id: 3, name: "Nitin", subjects: ["Math", "History"] }
])
```

Array Expression Operators

- **\$size:** Returns the number of elements in an array.

```
db.students.aggregate([ { $project: { numberOfSubjects: { $size: "$subjects" } } } ])
```

This will return the number of subjects for each student.
- **\$arrayToObject:** Converts an array into a single document.

```
db.students.aggregate([ { $project: { subjectsObject: { $arrayToObject: { $map: {
input: "$subjects", as: "subject", in: { k: "$$subject", v: { name: "$$subject", score: 0
}}}}}} } ])
```

Update operations in MongoDB

In MongoDB, update operations are used to modify existing documents in a collection. There are several methods to perform update operations:

- **updateOne():** Updates a single document that matches the specified filter. If multiple documents match the filter, only the first one found is updated.

```
db.collection('myCollection').updateOne(
    { _id: ObjectId('document_id') },
    { $set: { key: value } }
);
```
- **updateMany():** Updates all documents that match the specified filter.

```
db.collection('myCollection').updateMany(
    { key: value },
    { $set: { newKey: newValue } }
);
```

- **replaceOne():** Replaces a single document that matches the specified filter. The entire document is replaced with the new document.

```
db.collection('myCollection').replaceOne(  
  { key: value },  
  { newDocument }  
);
```

- **update():** Deprecated in recent versions of MongoDB, replaced by updateOne() and updateMany(). This method can update a single document or multiple documents.

```
db.collection('myCollection').update(  
  { key: value },  
  { $set: { newKey: newValue } }  
);
```

In these examples, db.collection('myCollection') specifies the collection you want to update. The \$set operator is used to specify the fields to update and their new values.

Array Update Modifiers:

- **\$:** Acts as a placeholder for the first element that matches the query condition in an update operation on an array field.
- **\$[]:** Acts as a placeholder for all elements that match the query condition in an update operation on an array field.
- **\$[<identifier>]:** Acts as a placeholder for the elements that match the arrayFilters condition in an update operation on an array field.

Remember to use the correct filters to target the documents you want to update. Incorrect filters can result in unintended updates or no updates at all.

Delete operations in MongoDB

In MongoDB, delete operations are used to remove documents from a collection. There are several methods to perform delete operations:

- **deleteOne():** Deletes a single document that matches the specified filter. If multiple documents match the filter, only the first one found is deleted.

```
db.collection('myCollection').deleteOne({ key: value });
```

- **deleteMany():** Deletes all documents that match the specified filter.

```
db.collection('myCollection').deleteMany({ key: value });
```

- **findOneAndDelete():** Finds a single document that matches the specified filter, deletes it, and returns the deleted document.

```
db.collection('myCollection').findOneAndDelete({ key: value });
```

- **delete():** Deprecated in recent versions of MongoDB, replaced by deleteOne() and deleteMany(). This method can delete a single document or multiple documents.

```
db.collection('myCollection').delete({ key: value });
```

In these examples, `db.collection('myCollection')` specifies the collection you want to delete documents from. The `{ key: value }` filter specifies which documents to delete.

Remember to use the correct filters to target the documents you want to delete. Incorrect filters can result in unintended deletions or no deletions at all.

Join us for Industry Based & Job Oriented Training Programs

Top Edge Training Programs



**MERN / MEAN Stack
Development**



**Java Full Stack
Development**



**Data Science
& Machine Learning**



**Data Analytics &
Business Intelligence**



**Linux, Cloud
& DevOps**



**Digital Marketing &
Business Development**

Amenities

- Digital Notes
- Learning References
- Live Training Sessions
- Live Project Training
- Interview Preparation
- Live Session Recordings for Revision
- Job Referrals & Recommendations
- Project Work Experience
- Professional Development
- Digital Resume & Portfolio

LEARN-2-EARN LABS TRAINING INSTITUTE

Head Office : F-4, First Floor, Anna Icon Complex, near Kargil Petrol Pump,
Sikandra-Bodla Road, Sikandra, Agra, Uttar Pradesh - 282007

Website : www.LearntoearnLabs.com

Call : 91-9548868337 / +91-9837705705

Data modelling in MongoDB

Data modeling in MongoDB involves designing the structure of your documents to best fit your application's needs. Here's a brief overview:

- **Document Structure:** MongoDB stores data in flexible, JSON-like documents, allowing for nested data structures. Each document can have a different structure, unlike relational databases where all rows in a table have the same structure.
- **Schema Design:** MongoDB's flexible schema design allows you to optimize your data model for your application's specific use cases. You can embed related data within a single document or reference it across multiple documents.
- **Embedding vs. Referencing:**
 - **Embedding:** Embedding related data within a single document can improve read performance, as you can retrieve all related data with a single query. However, it can lead to data duplication and potentially larger documents.
 - **Referencing:** Referencing related data across multiple documents can reduce data duplication and document size. However, it requires additional queries to retrieve related data, which can impact performance.
- **Indexing:** MongoDB supports indexes to improve query performance. You can create indexes on fields that are frequently queried or used for sorting to speed up read operations.
- **Normalization and Denormalization:** MongoDB allows you to normalize or denormalize your data based on your application's requirements. Normalization involves splitting data into separate collections to reduce data duplication, while denormalization involves combining related data into a single document for faster reads.
- **Aggregation Framework:** MongoDB provides an aggregation framework for performing complex queries, data transformations, and analysis on your data. It allows you to aggregate data from multiple documents and collections.
- **Data Consistency:** MongoDB provides strong consistency guarantees by default, ensuring that data is consistent across replicas in a replica set. However, you can configure read preferences and write concerns to balance consistency and performance based on your application's requirements.

Overall, data modeling in MongoDB involves understanding your application's requirements and designing a data model that balances performance, scalability, and data consistency.

Flat schema in MongoDB

In MongoDB, a flat schema refers to a data model where related data is stored in a single document without nesting or embedding. This approach is similar to how data is stored in relational databases, where each table represents a distinct entity, and relationships between entities are established through foreign keys.

In a flat schema in MongoDB:

- Each document represents a single entity or object.
- Related data is stored in separate collections, and relationships between entities are established using references (e.g., storing the `_id` of one document in another document to establish a relationship).
- There is no nesting or embedding of documents within other documents.

While MongoDB's flexible schema allows for nesting and embedding of documents, a flat schema can be beneficial in certain scenarios:

- **Performance:** Flat schemas can improve read and write performance for certain types of queries, especially when querying for data across multiple entities.
- **Simplicity:** Flat schemas can be easier to understand and maintain, especially for developers familiar with relational databases.
- **Query Flexibility:** Flat schemas allow for more flexibility in querying and aggregating data, as relationships are established through references rather than embedded documents.

let's consider an example where we have two entities: users and orders. We'll use a flat schema approach to model these entities in MongoDB.

Users Collection:

```
{
  "_id": ObjectId("user_id"),
  "name": " Meenakshee Jain ",
  "email": "meena121@example.com",
  "address": "Sanganer, Jaipur, Rajasthan"
  "phone": "123-456-7890"
}
```

Orders Collection:

```
{  
  "_id": ObjectId("order_id"),  
  "user_id": ObjectId("user_id"),  
  "order_date": "2 September 2021",  
  "total_amount": 100.50,  
  "status": "completed"  
}
```

In the example, we have a users collection where each document represents a user with fields such as name, email, address, and phone number. We also have an orders collection where each document represents an order with fields such as user_id (referencing the _id of the user document), order_date, total_amount, and status.

This approach allows us to query for orders and users separately and establish relationships between them using the user_id field in the orders collection. While this is a simple example, in a real-world scenario, you might have more complex relationships and additional collections to represent different entities in your application.

Star schema in MongoDB

Star schema is a data modeling technique used in data warehousing to organize data into a central "fact" table and multiple "dimension" tables. In MongoDB, which is a NoSQL database, the concept of star schema can be implemented using collections and document references.

In a star schema:

- **Fact Table:** This table contains the primary data that you want to analyze (e.g., sales transactions, orders, etc.). Each record in the fact table typically contains keys (references) to one or more dimension tables and measures (quantitative data).
- **Dimension Tables:** These tables contain descriptive attributes related to the data in the fact table. For example, if the fact table contains sales data, dimension tables might include tables for products, customers, time (e.g., dates), etc. Each dimension table typically has a primary key that is referenced in the fact table.

In MongoDB, you can represent a star schema using collections for fact and dimension tables, and document references for relationships between them. For example:

Fact Collection (e.g., "sales"):

```
{
  "_id": ObjectId("fact_id"),
  "date": "2 September 2021",
  "product_id": ObjectId("product_id"),
  "customer_id": ObjectId("customer_id"),
  "quantity": 10,
  "amount": 100.50
}
```

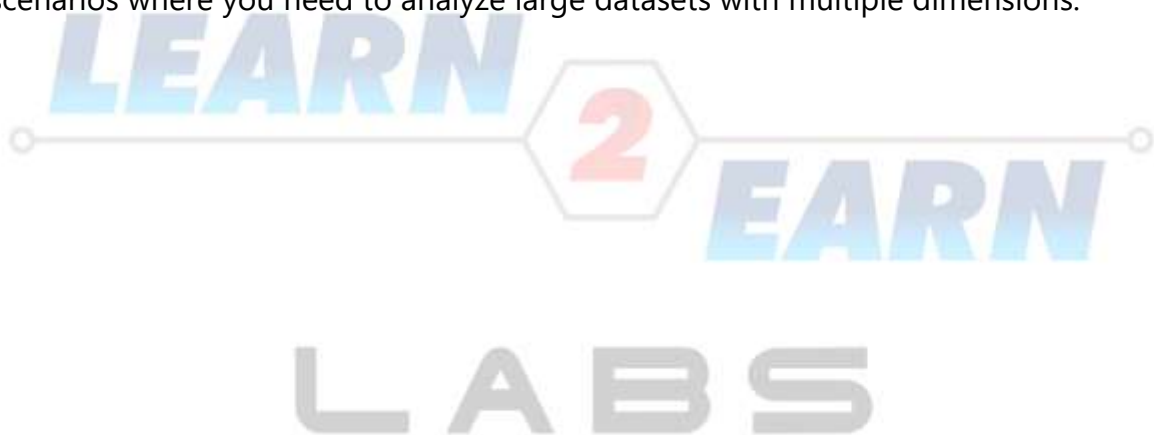
Dimension Collections (e.g., "products", "customers"):

```
// Products Collection
{
  "_id": ObjectId("product_id"),
  "name": "Product A",
  "category": "Category A"
}
```

```
// Customers Collection
{
  "_id": ObjectId("customer_id"),
  "name": " Meenakshee Jain ",
  "email": "meena121@example.com",
  "address": "Sanganer, Jaipur, Rajasthan"
}
```

In the example, the "sales" collection serves as the fact table, containing references (product_id and customer_id) to the "products" and "customers" collections, which act as dimension tables. This schema allows you to perform analytics and queries on the fact data while easily joining with the dimension data for additional context.

Using a star schema in MongoDB can help organize and query data efficiently, especially in scenarios where you need to analyze large datasets with multiple dimensions.



Hierarchical schema in MongoDB

A hierarchical schema in MongoDB is a way of organizing data where each document can have a parent-child relationship with other documents in the same collection. This approach is useful for representing data that has a natural hierarchical structure, such as organizational charts, file systems, or category hierarchies.

In a hierarchical schema:

- **Parent-Child Relationship:** Each document can have a reference to its parent document, creating a tree-like structure. For example, a document representing a category can have a reference to its parent category.
- **Single Collection:** All documents are stored in a single collection, and the hierarchical relationships are represented using references (e.g., `parent_id` field).
- **Traversal:** You can traverse the hierarchy by following the parent references from child to parent or vice versa.
- **Tree Structures:** The data can be structured as a tree, where each document can have multiple children but only one parent.

Example of a hierarchical schema in MongoDB for representing categories:

```
{
  "_id": ObjectId("category_id"),
  "name": "Electronics",
  "parent_id": null,
  "children": [
    {
      "_id": ObjectId("sub_category_id"),
      "name": "Mobile Phones",
      "parent_id": ObjectId("category_id"),
      "children": []
    },
    {
      "_id": ObjectId("sub_category_id"),
      "name": "Laptops",
      "parent_id": ObjectId("category_id"),
      "children": []
    }
  ]
}
```

In the example, the "Electronics" category is the root node with two child categories: "Mobile Phones" and "Laptops".

Each child category document has a reference (`parent_id`) to its parent category. This structure allows you to represent and query hierarchical data efficiently in MongoDB.

Relational schema in MongoDB

In MongoDB, a relational schema refers to a data modeling approach that resembles traditional relational database schemas, where data is organized into tables with rows and columns, and relationships between tables are established using foreign keys.

While MongoDB is a NoSQL database and does not enforce a fixed schema like relational databases, you can still design your MongoDB schema to mimic a relational schema for certain use cases.

Implement a relational schema in MongoDB

- **Collections as Tables:** Use MongoDB collections to represent tables in a relational schema. Each collection would store documents that correspond to rows in a table.
- **Documents as Rows:** Each document in a collection represents a row in a table. The fields within the document correspond to columns in the table.
- **References as Foreign Keys:** Use references within documents to establish relationships between collections. For example, a document in one collection might contain the `_id` of a document in another collection to represent a foreign key relationship.

Example: Consider a simple relational schema with two tables: users and orders. You can represent this schema in MongoDB as two collections:

Users Collection:

```
{
  "_id": ObjectId("user_id"),
  "name": " Meenakshee Jain ",
  "email": "meena121@example.com",
}
```

Orders Collection:

```
{
  "_id": ObjectId("order_id"),
  "user_id": ObjectId("user_id"),
  "order_date": "2 September 2021",
  "total_amount": 100.50
}
```

In the example, the `user_id` field in the `orders` collection serves as a foreign key referencing the `_id` field in the `users` collection, establishing a relationship between the two collections similar to a relational database.

While MongoDB's flexible schema allows for more complex and dynamic data structures than traditional relational databases, you can still design your schema to resemble a relational schema if it better suits your application's needs.

Object Relational Schema in MongoDB

In MongoDB, an Object-Relational Schema refers to a schema design approach that combines features of both relational databases and MongoDB's document-oriented data model. This approach is useful when you want to leverage MongoDB's flexibility while still maintaining some relational database-like characteristics.

Key aspects of an Object-Relational Schema in MongoDB include:

- **Normalization and Denormalization:** You can choose to normalize your data to reduce redundancy and improve data integrity, similar to relational databases. Alternatively, you can denormalize your data for better performance and to simplify querying, similar to MongoDB's document embedding.
- **References and Relationships:** You can use references to establish relationships between documents, similar to foreign keys in relational databases. This allows you to model complex relationships between entities.
- **Hybrid Schema Design:** You can design your schema to have a mix of normalized and denormalized data, depending on your application's requirements. For example, you might normalize certain entities that have many-to-many relationships while denormalizing others for better performance.
- **Querying:** MongoDB's query language allows you to perform complex queries across related documents, similar to SQL joins in relational databases. However, the approach and syntax may be different due to the document-oriented nature of MongoDB.
- **Flexibility:** MongoDB's flexible schema allows you to evolve your schema over time, adding or removing fields as needed, without requiring a predefined schema like in relational databases.

Let's consider an example of an Object-Relational Schema in MongoDB for a simple e-commerce application with users, products, and orders.

Normalized Schema:

Users Collection:

```
{
  "_id": ObjectId("user_id"),
  "name": " Meenakshee Jain ",
  "email": "meena121@example.com",
  "address_id": ObjectId("address_id")
}
```

Addresses Collection:

```
{
  "_id": ObjectId("address_id"),
  "street": "Sanganer",
  "city": "Jaipur",
  "state": "Rajasthan"
}
```

Products Collection:

```
{
  "_id": ObjectId("product_id"),
  "name": "Product A",
  "price": 100.00
}
```

Orders Collection:

```
{
  "_id": ObjectId("order_id"),
  "user_id": ObjectId("user_id"),
  "product_id": ObjectId("product_id"),
  "quantity": 2,
  "order_date": "2 September 2021"
}
```

Denormalized Schema:

Users Collection:

```
{
  "_id": ObjectId("user_id"),
  "name": " Meenakshee Jain ",
  "email": "meena121@example.com",
  "address": {
    "street": "Sanganer",
    "city": "Jaipur",
    "state": "Rajasthan"
  }
}
```

Products Collection:

```
{
  "_id": ObjectId("product_id"),
  "name": "Product A",
  "price": 100.00
}
```

Orders Collection:

```
{
  "_id": ObjectId("order_id"),
  "user": {
    "_id": ObjectId("user_id"),
    "name": "Meenakshee Jain"
  },
  "product": {
    "_id": ObjectId("product_id"),
    "name": "Product A",
    "price": 100.00
  },
  "quantity": 2,
  "order_date": "2 September 2021"
}
```

In the normalized schema, we have separate collections for users, addresses, products, and orders, with references (IDs) between them. This approach allows for better data integrity but may require additional queries to fetch related data.

In the denormalized schema, we have embedded the address information in the users collection and duplicated some product information in the orders collection. This approach simplifies querying but can lead to data redundancy and potentially larger document sizes.

The choice between normalized and denormalized schemas depends on the specific requirements of your application, including the balance between data integrity, performance, and scalability.



Join us for Industry Based & Job Oriented Training Programs

Top Edge Training Programs



**MERN / MEAN Stack
Development**



**Java Full Stack
Development**



**Data Science
& Machine Learning**



**Data Analytics &
Business Intelligence**



**Linux, Cloud
& DevOps**



**Digital Marketing &
Business Development**

Amenities

- Digital Notes
- Learning References
- Live Training Sessions
- Live Project Training
- Interview Preparation
- Live Session Recordings for Revision
- Job Referrals & Recommendations
- Project Work Experience
- Professional Development
- Digital Resume & Portfolio

LEARN-2-EARN LABS TRAINING INSTITUTE

Head Office : F-4, First Floor, Anna Icon Complex, near Kargil Petrol Pump,
Sikandra-Bodla Road, Sikandra, Agra, Uttar Pradesh - 282007

Website : www.LearntoearnLabs.com

Call : 91-9548868337 / +91-9837705705

Distributed database in MongoDB

In MongoDB, a distributed database refers to a database system that spans multiple nodes or servers to store and manage data. MongoDB achieves distribution through sharding, which involves dividing data into smaller chunks called shards and distributing them across multiple servers. This allows MongoDB to handle large amounts of data and provide high availability and scalability.

How distributed databases work in MongoDB

Imagine you are building an e-commerce platform that needs to handle a large number of products and orders. To ensure scalability and high availability, you decide to use MongoDB's distributed database capabilities.

- **Sharding:** You start by setting up a MongoDB cluster with three shards, each running on a separate server or replica set. You choose to shard the data based on the product category, so each shard will contain data for a specific category of products.
 - Shard 1:** Contains data for electronics products.
 - Shard 2:** Contains data for clothing products.
 - Shard 3:** Contains data for home appliances products.
 - Shard Key:** The shard key for this setup would be the product category. MongoDB will use this key to determine which shard should store each product document based on its category.
- **Balancing:** MongoDB automatically balances the data across shards to ensure that each shard has a similar amount of data. For example, if the electronics shard becomes too large, MongoDB will move some electronics products to the other shards to balance the load.
- **High Availability:** Each shard is configured as a replica set, with multiple copies of the data (primary and secondary nodes). If a primary node fails, MongoDB will elect a new primary from the remaining nodes to ensure continuous availability.
- **Scaling:** As your platform grows and you need to handle more products and orders, you can add more shards to the cluster. MongoDB allows you to dynamically add shards to the cluster without downtime, ensuring that your database can scale to meet the demands of your application.

In this example, MongoDB's distributed database capabilities enable you to store and manage a large amount of data efficiently, while ensuring high availability and scalability for your e-commerce platform.

Indexes in MongoDB

- Indexes in MongoDB are data structures that store a small portion of the collection's data set in an easy-to-traverse form.
- They support the efficient execution of queries in MongoDB by allowing the database to quickly locate and access the documents that match the criteria specified in the query.
- Indexes improve query performance by reducing the number of documents that need to be scanned.
- Indexes play a crucial role in optimizing query performance in MongoDB by facilitating efficient data retrieval and traversal. However, it's essential to carefully design and manage indexes to balance performance improvements with storage and write performance considerations.

Key points about indexes in MongoDB

- **Types of Indexes:** MongoDB supports various types of indexes, including single field, compound, multikey, text, geo-spatial and hashed indexes. Each type of index is designed to optimize query performance for specific use cases.
- **Creating Indexes:** Indexes can be created using the `createIndex()` method in MongoDB. You can specify which fields to index and the type of index to create. Indexes can be created either in the background or in the foreground, depending on whether you want the creation process to block other operations on the database.
- **Index Usage:** MongoDB's query optimizer automatically selects the most efficient index to use for a query based on the query's execution plan and the available indexes on the collection. You can use the `explain()` method to analyze the query execution plan and determine which indexes are being used.
- **Index Size and Performance:** While indexes improve query performance, they also consume additional storage space and can impact write performance during insert, update, and delete operations. It's essential to consider the trade-offs between query performance and storage space when creating indexes.
- **Index Management:** MongoDB provides methods for managing indexes, such as dropping indexes using the `dropIndex()` method and listing existing indexes using the `getIndexes()` method. You can also configure index options, such as uniqueness constraints and index collation, when creating indexes.

- **Index Strategies:** MongoDB supports various index strategies to optimize query performance, such as covering indexes, compound indexes, and partial indexes. Understanding these strategies and choosing the appropriate indexes for your queries can significantly improve performance.

Example

First you need to create a json file with data, so you can refer

<https://jsonplaceholder.typicode.com/photos>

and use the following command in command prompt to import the database

```
mongoimport --db media --collection photos --file "E:\json data\photos.json" --jsonArray
```

explain()

- In MongoDB, the `explain()` function is a powerful tool used to analyze query performance and understand how MongoDB executes a particular query. It provides detailed information about the query execution plan, including the selected indexes, the number of documents examined, and various performance metrics.
- Using `explain('executionStats')` allows you to gain deeper insights into query performance and understand the factors influencing query execution. It helps you identify potential bottlenecks, optimize query performance, and make informed decisions about index creation, query structure, and database schema design.

Now use the following commands in MongoShell

```
use media
db.photos.find().explain()
db.photos.find().explain('executionStats')
db.photos.find({id:{$gt:50}}).explain('executionStats')
db.photos.find({id:{$gt:2550}}).explain('executionStats')
```

and analyze the

```
executionStats: {
  nReturned: xx,
  executionTimeMillis: xx,
  totalDocsExamined: xx,
  stage:xx    }
```

For all the collections inside the databases, MongoDB automatically define `_id` as index.

To **get the details about the collection's index** use the following command

```
db.collectionName.getIndexes()
```

and you will get the something like:

```
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
```

Where,

- **{ v: 2 }:** This indicates the index version. In this case, v: 2 corresponds to index version 2. MongoDB has multiple index versions, and each version may have different characteristics or optimizations.
- **key: { _id: 1 }:** This specifies the fields included in the index and their sort order. In this case, it indicates that the index is built on the `_id` field in ascending order (1 signifies ascending order, -1 would indicate descending order).
- **name: '_id_':** This is the name of the index. In this example, the index is named `_id_`. MongoDB automatically creates an index on the `_id` field for each collection to ensure its uniqueness and facilitate efficient document retrieval.

To **create a new index**, use the following command in mongo shell

```
db.collectionName.createIndex{field:sortorder}
```

for example:

```
db.photos.createIndex({albumId:1})
```

Now, you will get two indexes for your collection, using the command

```
db.photos.getIndexes()
```

& If you try to get the details of query performance based on the newly created index using the below command, then you will get the different information

```
db.photos.find({albumId:47}).explain('executionStats')
```

previously the stage was COLLSCAN (collection scan) and now I will be IXSCAN (index scan)

To **drop or delete the created index**, use the following command in mongo shell

```
db.photos.dropIndex('albumId_1')
```

Now, if you want to get the indexes then you will only get one index for your collection

```
db.photos.getIndexes()
```

note : It's important to note that dropping an index may impact query performance, so you should carefully consider the consequences before removing an index. Additionally, dropping an index is a blocking operation, which means that it may temporarily affect the performance of other operations on the collection while the index is being dropped.

When not to use Indexes

While indexes can significantly improve query performance in MongoDB, there are scenarios where they may not be required or may not provide significant benefits.

Here are some situations where indexes might not be necessary:

- **Small Collections:** In small collections with only a few hundred or thousand documents, the overhead of maintaining indexes may outweigh the performance benefits. Queries on small collections can be fast even without indexes because MongoDB can quickly scan through all documents.
- **Frequent Write Operations:** If a collection experiences frequent insertions, updates, or deletions, maintaining indexes can impose additional overhead on write operations. In such cases, you might prioritize write performance over read performance and choose to avoid unnecessary indexes.
- **Full Collection Scans:** Some queries may need to retrieve a large portion or all documents from a collection. In such cases, using an index might not be beneficial as MongoDB may opt for a full collection scan instead of using the index.
- **Batch Processing:** For batch processing tasks or analytics queries that require scanning large portions of the collection, indexes might not be necessary. These queries may involve aggregations or complex calculations that don't benefit significantly from index usage.
- **Random Access Patterns:** If your application primarily performs random access patterns, such as accessing documents based on non-indexed fields or using a wide variety of query predicates, maintaining indexes for each query pattern might not be feasible or practical.
- **Temporary Queries:** If a query is only used sporadically or for a short period, creating an index solely for that query may not be worth the overhead. It's essential to evaluate the long-term usefulness of an index before creating it.
- **Resource Constraints:** In environments with limited resources, such as embedded systems or IoT devices, maintaining multiple indexes can consume significant storage space and memory. In such cases, you may need to prioritize resource usage and avoid unnecessary indexes.

- **Rarely used fields:** Indexes may not be required for rarely used fields, especially if those fields are not frequently queried or if the queries on those fields don't contribute significantly to the overall workload of the application.

In summary, while indexes are essential for optimizing query performance in many scenarios, it's essential to carefully consider whether they are necessary based on the specific requirements, workload characteristics, and resource constraints of your MongoDB deployment. Over-indexing can lead to increased storage requirements, slower write performance, and higher maintenance overhead, so it's crucial to strike the right balance between index usage and query optimization.

Aggregation Framework

- The MongoDB Aggregation Framework is a powerful tool for data aggregation and analysis, providing a way to process data from a collection and return computed results.
- It allows you to perform various operations, such as filtering, grouping, sorting, projecting, and computing aggregate functions, among others, on multiple documents within a collection.
- In MongoDB, aggregation refers to the process of performing operations on multiple documents within a collection to transform or analyze data.
- MongoDB provides the `aggregate()` method to perform aggregation operations.

Key concepts of MongoDB Aggregation Framework

- **Pipeline Concept**
 - Aggregation operations in MongoDB are performed using pipelines. A pipeline is a sequence of stages where each stage represents a different operation to be performed on the data.
 - Each stage in the pipeline processes the documents and passes the results to the next stage in the pipeline.
- **Stages**
 - MongoDB provides various stages that can be used in the aggregation pipeline. Some common stages include `$match`, `$group`, `$project`, `$sort`, `$limit`, and `$unwind`, among others.
 - Each stage performs a specific operation on the documents and can have its own set of parameters and options.

- **Operators**

- MongoDB offers a wide range of operators that can be used within the aggregation pipeline. These operators allow you to perform arithmetic operations, comparison operations, array operations, and more.
- Operators can be used within different stages to manipulate and transform data as needed.

- **Aggregation Functions**

- MongoDB provides several aggregation functions that can be used within the aggregation pipeline to perform computations on the data.
- Common aggregation functions include \$sum, \$avg, \$min, \$max, \$count, \$group, \$addToSet, and \$push, among others.

MongoDB aggregate pipeline syntax

This is an example of how to build an aggregation query:

`db.collectionName.aggregate(pipeline, options),`

where

- `collectionName` – is the name of a collection,
- `pipeline` – is an array that contains the aggregation stages,
- `options` – optional parameters for the aggregation

This is an example of the aggregation pipeline syntax:

```
pipeline = [  
  { $match : { ... } },  
  { $group : { ... } },  
  { $sort : { ... } }  
]
```

Example: MongoDB's Aggregation Framework with an input JSON file.

`store.json`

```
[  
  { "product": "Apple", "quantity": 10, "price": 150, "date": "2024-03-01" },  
  { "product": "Banana", "quantity": 25, "price": 30, "date": "2024-03-01" },  
  { "product": "Apple", "quantity": 8, "price": 120, "date": "2024-03-02" },  
  { "product": "Orange", "quantity": 12, "price": 80, "date": "2024-03-02" },  
  { "product": "Banana", "quantity": 7, "price": 40, "date": "2024-03-03" }  
]
```


To calculate the total revenue generated each day, use the below **MongoDB code**

```
db.store.aggregate([
  { $group: {
    _id: "$date",    // Group by date
    totalRevenue: { $sum: { $multiply: ["$quantity", "$price"] } }
    // Calculate total revenue for each date
  } },
  { $sort: { _id: 1 } } // Sort by date in ascending order
])
```

To find out the total sales quantity for each product, use the below **MongoDB code**

```
db.store.aggregate([
  { $group: {
    _id: "$product", // Group by product
    totalQuantity: { $sum: "$quantity" } // Calculate total quantity for each product
  } }
])
```

To find the average price of each product, use the below **MongoDB code**

```
db.store.aggregate([
  { $group: {
    _id: "$product", // Group by product
    averagePrice: { $avg: "$price" } // Calculate average price for each product
  } }
])
```

Working with Large Objects

Working with large objects, such as binary data or large documents, in MongoDB can be challenging due to document size limitations and performance considerations. Here are some strategies for working with large objects in MongoDB:

- **GridFS for Large Binary Data**

- GridFS is a specification for storing and retrieving large binary files in MongoDB. It divides the large file into smaller chunks, which are then stored as separate documents in two collections: `fs.files` and `fs.chunks`.
- Use GridFS when storing large files, such as images, videos, or documents, in MongoDB.
- GridFS allows for efficient retrieval and streaming of large files without loading the entire file into memory.

- **Large Documents Optimization**

- MongoDB has a document size limit of 16 MB. If your documents approach or exceed this limit, consider redesigning your data model.
- Break down large documents into smaller, more manageable chunks.
- Use references (e.g., `$ref` fields) to other documents instead of embedding large amounts of data within a single document.

- **Streaming Data**

- When dealing with large objects, consider streaming data rather than loading the entire object into memory at once.
- Use MongoDB cursors (`cursor.forEach()` or `cursor.next()`) to iterate over large result sets without loading all documents into memory simultaneously.

- **Indexing**

- Create appropriate indexes on fields used for querying large objects to improve query performance.
- Consider indexing fields used for sorting or grouping large datasets.

- **Memory Considerations**

- Be mindful of memory usage, especially when working with large documents or result sets.
- Adjust MongoDB's `storage.wiredTiger.engineConfig.cacheSizeGB` setting to allocate appropriate memory for caching data.

- **Compression and Chunking**

- Consider compressing large objects before storing them in MongoDB to reduce storage space and improve performance.
- For extremely large objects, you may need to manually split them into smaller chunks and store them separately.

- **Sharding**

- If your dataset grows too large to fit on a single MongoDB instance, consider sharding your database to distribute data across multiple servers.
- Sharding helps distribute the load and allows MongoDB to scale horizontally to handle large datasets.

- **Streaming and Pagination**

- When querying large datasets, use streaming and pagination techniques to retrieve and process data in smaller batches.
- Limit the number of documents returned per query using `limit()` and `skip()` methods.

By employing these strategies, you can effectively work with large objects in MongoDB while ensuring optimal performance and scalability.

LABS



LEARN-2-EARN LABS TRAINING INSTITUTE

F-4, First Floor, Anna Icon Complex,
near Kargil Petrol Pump, Sikandra-Bodla Road,
Sikandra, Agra, Uttar Pradesh, India

Website : www.LearntoearnLabs.com

Call : 91-9548868337 / +91-9837705705